INTERVIEW PREPARATION GUIDE

Frontend Developer WeMakeScholars

React · Next.js · Performance & SEO · Machine Coding · HR
Prepared for Sujith Mondithoka · Interview: 10:00 AM

How to use this guide. Work the sections in order. The plan on the next page is timed — if you fall behind, drop from the bottom, never from the top.

● **Red dot** = high priority, almost certainly asked. ●● = your differentiator. ● **Amber** = worth knowing.

✓ = self-check before moving on. ⚠ = common trap or something to verify.

Contents

WeMakeScholars — Frontend Developer Interview Prep

Your plan for tonight and tomorrow morning · Interview: 10:00 AM

How this guide is written

Every topic follows the same shape, so you always know what you are reading:

1. **The context** — what the thing is, and what problem it solves, in plain words.
2. **A simple example** — small code you can actually follow.
3. **Why it matters** — where it is used in real work.
4. **"Say this"** — the answer to give in the room.

You are not meant to memorise the answers. You are meant to understand the idea, so that when the question comes out slightly differently, you can still answer it.

1. What the job description is really telling you

I have read a lot of these. Here is what each line means for your interview.

The JD says	What it means for you
"React and Next.js"	Next.js is not optional. They will ask about rendering strategies. **File 03 is your biggest opportunity.**
"Making Things Fast" *(they highlighted this in their own document)*	The person who wrote the JD cares about it. Learn Core Web Vitals. **File 04.**
"look great on every device"	Responsive CSS, Flexbox and Grid, mobile first. **File 05.**
"perform well on search engines"	SEO is how this company gets customers. Server rendering, metadata, semantic HTML. **Files 03 and 04.**
"plug REST APIs and data into the frontend"	fetch and axios, loading and error states, race conditions. **File 06.**
"reusable components, readable, maintainable code"	They will ask how you would design a component, and probably give you a coding task. **Files 07 and 08.**
"Partnering with designers and product"	Behavioural round. Have real stories ready. **File 09.**
18 month commitment, 2 month notice, work from office, 1st and 3rd Saturdays	These will be asked as filter questions. Scripted answers in **File 09.**

The one thing that will set you apart

Almost everyone applying for a 4 to 6 LPA frontend role can **use** React. Very few can explain **why Next.js exists** and **what problem server rendering solves for a business that depends on Google search**.

WeMakeScholars finds students through organic search. If your technical answers connect back to *"this helps your pages get found and load fast for students on slow phones"*, you stop sounding like a bootcamp graduate and start sounding like an engineer.

Say it **once**, at the right moment. Not five times.

---

2. What the interview will probably look like

For a 250 person company hiring at this level, expect 2 or 3 rounds, often on the same day.

Round 1 — Technical screening (45 to 60 min).
JavaScript fundamentals, React hooks, "explain your project", some CSS.
→ Files 01, 02, 05, 10.

Round 2 — Machine coding or a live task (45 to 60 min).
Build a small component. Usually a search and filter list, a form with validation, or fetching and displaying data.
→ File 07.

Round 3 — Tech lead or manager (30 min).
Next.js, performance, SEO, system design, how you work with backend and design.
→ Files 03, 04, 06, 08.

Round 4 — HR (20 min).
Commitment, notice period, salary, working Saturdays.
→ File 09.

---

3. The schedule — from now until 10:00 AM tomorrow

Time is your scarce resource, so this is ordered by importance. **If you fall behind, drop things from the bottom, never from the top.**

Today, afternoon and evening

Time	What	File
0:00 – 0:15	Read this file	`README.md`
0:15 – 1:30	**React** 🔴	`02-react.md`
1:30 – 2:45	**Next.js** 🔴🔴	`03-nextjs.md`

Time	What	File
2:45 – 3:00	Break. Walk around. No phone.	—
3:00 – 4:00	**JavaScript** 🔴	`01-javascript.md`
4:00 – 5:00	**Performance and SEO** 🔴	`04-performance-seo.md`
5:00 – 5:45	Dinner. Actually eat properly.	—
5:45 – 6:45	**Machine coding** 🔴 — type the code, do not read it	`07-machine-coding.md`
6:45 – 7:30	**System design** 🔴	`08-system-design.md`
7:30 – 8:15	**Your projects** — say the pitch out loud	`10-your-projects.md`
8:15 – 9:00	**HR answers** — say these out loud too	`09-hr-and-behavioural.md`
9:00 – 9:30	REST APIs, quick read	`06-rest-api-integration.md`
By 11:30 PM	**Sleep.** This is not optional.	—

> **Studying past midnight will make you worse, not better.** A tired candidate blanks on things they knew perfectly at 1 AM. Go to bed.

Tomorrow morning

Time	What	File
07:00 – 07:20	Wake up, shower, coffee. No scrolling.	—
07:20 – 08:00	**Cheat sheet only**	`11-final-cheatsheet.md`
08:00 – 08:30	HTML and CSS, quick read	`05-html-css-responsive.md`
08:30 – 09:00	Say your intro and project pitch **out loud, twice**	`10-your-projects.md`
09:00 – 09:20	Re-read the questions you will ask them	`09-hr-and-behavioural.md` §6
09:20 – 09:40	Check laptop, charger, internet, ID, resume copies, water	—
09:40 – 09:55	Close everything. Sit up straight. Breathe.	—
10:00	**Go.**	—

Rule for tomorrow morning: do not learn anything new. New material after 7 AM only pushes out what you already know and makes you anxious. Revision only.

---

4. The files

File	Topic	Priority
`01-javascript.md`	Closures, event loop, async, `this`, ES6, debounce	🔴 High
`02-react.md`	Hooks, re-renders, keys, context, controlled inputs	🔴 Highest
`03-nextjs.md`	CSR / SSR / SSG / ISR, App Router, Server Components, SEO	🔴 Highest
`04-performance-seo.md`	Core Web Vitals, how to diagnose slowness, SEO checklist	🔴 High
`05-html-css-responsive.md`	Semantic HTML, box model, Flexbox vs Grid, mobile first	🟠 Medium
`06-rest-api-integration.md`	fetch vs axios, the four states, race conditions, CORS	🟠 Medium
`07-machine-coding.md`	8 coding problems with full working solutions	🔴 High
`08-system-design.md`	How to answer design questions, with 8 worked examples	🔴 High
`09-hr-and-behavioural.md`	Filter questions, salary, STAR stories, questions to ask	🔴 High
`10-your-projects.md`	How to talk about your own GitHub projects	🔴 High
`11-final-cheatsheet.md`	One page. Tomorrow morning only.	🔴 Read last

There are also PDF versions in `pdf/`. The cheat sheet is designed to be printed.

---

5. Three rules for the interview room

1. Never just say "I don't know" and stop.
Say this instead:

> "I have not used that directly. My understanding is that it does X. Is that the direction you mean?"

Interviewers hire people who can reason out loud. They do not hire people who can only recite.

2. Keep talking during the coding round.
Silence makes them think you are stuck. Narrate what you are doing:

> "I will hold the input in state, debounce it, then filter the list..."

3. Connect one answer to their business.

> "Since students find you through Google, server rendering matters more here than it would in an internal dashboard."

One sentence like that is what makes an interviewer remember you out of forty CVs.

---

6. A note about your resume

Your resume PDF is on your own laptop, which this session cannot read. So `10-your-projects.md` was built from your **public GitHub repositories** instead.

Open that file and check it against your actual resume. The interviewer will have your PDF in front of them, so the two must match.

01 · JavaScript Core

Time needed: 60 minutes

Every frontend interview starts here. React and Next.js are both written in JavaScript. If your basics are weak, the interviewer will stop early and the rest of the interview will not happen.

How to read this file: each question has four parts.
What it means (the idea in plain words) → *Simple example* (small code) → *Why it matters* (the real problem it solves) → *Say this* (your answer in the room).

First, the context: what is JavaScript doing on a web page?

A web page has three parts.

Part	Language	Job
Structure	HTML	What is on the page. Headings, buttons, forms.
Look	CSS	Colours, spacing, layout.
Behaviour	**JavaScript**	What happens when the user does something.

HTML and CSS cannot make decisions. They cannot fetch data. JavaScript can. So when a student clicks "Check loan offers", JavaScript is the part that reads the form, calls the server, waits for the answer, and updates the page.

JavaScript runs **inside the browser**. Every browser has a JavaScript engine built in. This is why it is the only language that runs on a web page directly.

Q1. What is the difference between `var`, `let` and `const`? ●

What it means

These are three ways to create a variable. A variable is a name for a value.

```
var  a = 1;   // old way, from 1995
let  b = 2;   // new way, when the value will change
const c = 3;  // new way, when the value will not change
```

The difference that matters: scope

Scope means "which part of the code can see this variable".

`let` and `const` live only inside the nearest `{ }` block.
`var` ignores blocks. It leaks out to the whole function.

```
function test() {
  if (true) {
    var x = 'I leak';
    let y = 'I stay inside';
  }
  console.log(x);  // "I leak"  ← still visible, this is confusing
  console.log(y);  // ReferenceError ← y is gone, this is correct
}
```

The classic bug this causes

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100);
}
// prints 3, 3, 3   ← WRONG

for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100);
}
// prints 0, 1, 2   ← correct
```

With `var` there is only **one** `i` for the whole loop. By the time the timers run, the loop has finished and `i` is 3. With `let`, each turn of the loop gets its own fresh `i`.

Careful: `const` does not mean "cannot change"

`const` only means you cannot point the name at something new.
If the value is an object or array, the **inside** can still change.

```
const user = { name: 'Sujith' };
user.name = 'Kumar';        // <span class="tick">✓</span> allowed. We changed what is inside.
user = { name: 'Kumar' };   // ❌ error. We tried to point `user` somewhere new.
```

Say this

> " `var` is function scoped, so it leaks out of `if` blocks and loops. `let` and `const` are block scoped, so they stay inside the braces. I use `const` by default, `let` only when I need to reassign, and I do not use `var`. One thing to note is that `const` does not make an object immutable. It only stops you from reassigning the variable itself."

Q2. What is a closure? ●

What it means

A closure is a function that **remembers the variables around it**, even after the outer function has finished running.

Think of it like a lunchbox. When you create the inner function, it packs a copy of the surrounding variables. Wherever it goes later, it still has that lunchbox.

Simple example

```
function makeCounter() {
  let count = 0;              // this variable lives inside makeCounter

  return function () {
    count = count + 1;
    return count;
  };
}

const counter = makeCounter();
counter();  // 1
counter();  // 2
counter();  // 3
```

`makeCounter()` already finished on the first line. Normally `count` would be deleted. But the returned function still needs it, so JavaScript keeps it alive. That is a closure.

Notice something useful: nothing outside can touch `count`. You cannot write `counter.count = 100`. The variable is private.

Where you actually use closures

1. **Private data.** As above. The only way to change `count` is through the function.
2. `useState` **in React.** React stores your state in a closure and hands you back a value and a setter function.
3. **Debounce and throttle.** The timer id is kept in a closure between calls. (See Q10.)
4. **Event handlers that remember which item they belong to.**

The trap they may ask about: stale closure

```
useEffect(() => {
  const id = setInterval(() => {
    setCount(count + 1);   // ❌ `count` is stuck at its first value, 0
  }, 1000);
  return () => clearInterval(id);
}, []);                    // empty array = this runs once
```

The function inside `setInterval` packed its lunchbox on the first render only. It will always see `count` as 0. So the counter goes 0 → 1, 1, 1, 1 forever.

Fix: use the function form of the setter. It always receives the newest value.

```
setCount(c => c + 1);      // <span class="tick">✓</span>
```

Say this

> "A closure is a function that keeps access to the variables from where it was created, even after that outer function has returned. The counter example is the simplest one. In real code I use it for private state and for debounce, where the timer id has to survive between calls. It is also why the stale closure bug happens in `useEffect` with an empty dependency array."

Q3. What is the event loop? ●

The problem it solves

JavaScript can only do **one thing at a time**. It has a single call stack.

So what happens when you call an API that takes 2 seconds? If JavaScript simply waited, the whole page would freeze. No clicking, no scrolling, nothing.

It does not wait. This is what the event loop is for.

How it works, step by step

1. Your code runs on the **call stack**, one line at a time.
2. Slow things (network calls, timers) are **not** run by JavaScript. They are handed to the browser, which handles them in the background.
3. When the browser finishes, it puts your callback function in a **queue**.
4. The **event loop** watches the call stack. The moment the stack is empty, it takes the first item from the queue and runs it.

A simple picture:

```
your code  →  [ CALL STACK ]  ← event loop pushes callbacks back in
                                        ↑
slow work  →  [ browser handles it ] → [ QUEUE ]
```

The one detail interviewers test

There are two queues, and one has priority.

- **Microtask queue** — Promises, `async/await`. **Higher priority.**
- **Macrotask queue** — `setTimeout`, `setInterval`, click events. Lower priority.

All microtasks run before the next macrotask.

```
console.log('1');
setTimeout(() => console.log('2'), 0);
Promise.resolve().then(() => console.log('3'));
console.log('4');
```

Output: **1, 4, 3, 2**

Why:

- `1` and `4` are normal code, so they run first, in order.
- `3` is a Promise, so it goes to the microtask queue.
- `2` is a `setTimeout`, so it goes to the macrotask queue.
- Microtasks win, so `3` runs before `2`. Even though the timeout was 0 ms.

Say this

> "JavaScript is single threaded, so it runs one thing at a time. Slow work like network calls is given to the browser instead of blocking the stack. When it finishes, the callback waits in a queue, and the event loop pushes it back onto the stack once the stack is empty. The detail people miss is that promises go to the microtask queue, which is drained fully before any `setTimeout` runs."

Q4. Callbacks, Promises and async/await ●

The context: three generations of the same idea

All three solve one problem. *How do I run some code later, after a slow thing finishes?*

1. Callback (oldest). Pass a function in. It gets called when the work is done.

```
getUser(1, function (user) {
  getOrders(user.id, function (orders) {
    getDetails(orders[0], function (details) {
      console.log(details);        // three levels deep already
    });
  });
});
```

This shape is called **callback hell**. It grows sideways and is hard to read.

2. Promise. An object that represents a value you will get later.
It has three states: `pending` → then either `fulfilled` or `rejected`.

```
getUser(1)
  .then(user => getOrders(user.id))
  .then(orders => getDetails(orders[0]))
  .then(details => console.log(details))
  .catch(err => console.log('Something failed:', err));
```

Flat, not nested. One `.catch` handles errors from any step.

3. async/await (newest). The same Promise, written so it reads like normal code.

```
async function show() {
  try {
    const user    = await getUser(1);
    const orders  = await getOrders(user.id);
    const details = await getDetails(orders[0]);
    console.log(details);
  } catch (err) {
    console.log('Something failed:', err);
  }
}
```

`await` means "pause here until this promise finishes". It does not freeze the browser. It only pauses this one function.

Real fetch example, with the mistake people make

```
async function getLoans() {
  const res = await fetch('/api/loans');

  // <span class="warn"><span class="warn">⚠</span></span> fetch does NOT throw an error on 404 or
500.
  // It only throws if the network itself failed.
  // So you must check this yourself:
  if (!res.ok) {
    throw new Error('Request failed with status ' + res.status);
  }

  return res.json();
}
```

This catches many candidates. Remember it.

Running calls in parallel ●

```
// SLOW: 300ms + 300ms = 600ms
const banks  = await getBanks();
const offers = await getOffers();

// FAST: both start together, takes about 300ms
const [banks, offers] = await Promise.all([getBanks(), getOffers()]);
```

Use `Promise.all` when the calls do not depend on each other.

Method	What it does
`Promise.all`	Waits for all. If any one fails, the whole thing fails.
`Promise.allSettled`	Waits for all. Tells you which passed and which failed.
`Promise.race`	Returns the first one to finish, success or failure.
`Promise.any`	Returns the first one to **succeed**.

Say this

> "Callbacks came first but nesting them becomes unreadable. Promises fixed the nesting with chaining. `async/await` is the same promise written in a way that reads top to bottom, with normal `try/catch` for errors. One thing I always remember is that `fetch` does not reject on a 404 or 500, so I check `res.ok` myself. And when two calls do not depend on each other I use `Promise.all` so they run in parallel instead of one after the other."

Q5. What is `this`? How are arrow functions different? ●

What it means

`this` is a word that points to an object. Which object it points to is decided by **how the function is called**, not where it was written.

The four rules

```
const user = {
  name: 'Sujith',
  greet() { console.log(this.name); }
};

user.greet();          // "Sujith"   → called on user, so `this` = user

const g = user.greet;
g();                   // undefined  → called alone, so `this` is not user

g.call(user);          // "Sujith"   → we forced `this` to be user

new Person();          // `this` = the new object being created
```

Arrow functions are different

A normal function decides `this` when it is called.
An arrow function does **not** have its own `this`. It borrows it from the code around it, at the time it was written. This can never be changed.

```
const user = {
  name: 'Sujith',
  regular: function () { console.log(this.name); },
  arrow:   () => { console.log(this.name); }
};

user.regular();  // "Sujith"    ← this = user
user.arrow();    // undefined   ← this came from outside the object
```

Why this is useful

Before arrow functions, this was a common bug:

```
const timer = {
  seconds: 0,
  start: function () {
    setInterval(function () {
      this.seconds++;      // ❌ `this` is not `timer` here
    }, 1000);
  }
};
```

The fix used to be `.bind(this)`. Now we just use an arrow function:

```
setInterval(() => { this.seconds++; }, 1000);   // <span class="tick">✓</span> borrows `this` from start()
```

Say this

> "`this` depends on how a function is called. If you call `obj.method()`, `this` is `obj`. If you pull the same function out and call it alone, `this` is lost. Arrow functions do not have their own `this`. They take it from the surrounding code, which is why they solved the old `bind(this)` problem in callbacks. In React with hooks I rarely deal with `this` at all now."

Q6. ES6 features you use every day

These are just shortcuts. Learn what each one replaces.

Destructuring — pull values out of an object or array.

```
// instead of:
const name = user.name;
const city = user.city;

// write:
const { name, city } = user;
const [first, second] = myArray;
```

Default value — used when the value is `undefined`.

```
const { city = 'Hyderabad' } = user;
```

Spread `...` — copy an object or array into a new one.

```
const newUser  = { ...user, verified: true };   // copy + change one field
const newList  = [...list, newItem];            // copy + add one item
```

⚠ Spread makes a **shallow** copy. Objects inside are still shared.
For a full copy: `structuredClone(user)`.

Rest `...` — collect the remaining items.

```
const [first, ...others] = [1, 2, 3, 4];   // first = 1, others = [2, 3, 4]
```

Template literal — build a string without `+`.

```
const msg = `Hello ${name}, you have ${count} offers`;
```

Optional chaining `?.` — stop safely if something is missing.

```
const city = user?.address?.city;    // undefined instead of a crash
```

Nullish coalescing `??` — a fallback, but only for `null` and `undefined`.

```
const rate = apiRate ?? 8.5;
```

● `??` vs `||` — know this difference

```
const rate = 0;

rate || 8.5   // 8.5   ← wrong! 0 is falsy, so it was replaced
rate ?? 8.5   // 0     ← correct, 0 is a real value
```

`||` replaces **any** falsy value: `0`, `''`, `false`, `null`, `undefined`, `NaN`.
`??` replaces **only** `null` and `undefined`.

If an interest rate of 0% is valid, `||` will silently give you the wrong number.

Q7. `==` vs `===`, and `null` vs `undefined`

`===` compares value and type. No conversion.
`==` converts the types first, which gives strange results.

```
'5' === 5      // false   ← different types
'5' == 5       // true    ← '5' was converted to 5
0 == ''        // true
null == undefined  // true
null === undefined // false
```

Rule: always use `===`.

`undefined` — JavaScript's default. The variable exists but was never given a value.
`null` — you deliberately said "empty on purpose".

Falsy values, memorise the list:
`false`, `0`, `''` (empty string), `null`, `undefined`, `NaN`.
Everything else is truthy, including `[]` and `{}`.

Q8. Array methods: map, filter, reduce, forEach ●

These come up constantly, because React lists are built with `map`.

```
const loans = [
  { id: 1, bank: 'SBI',  amount: 500000 },
  { id: 2, bank: 'HDFC', amount: 800000 },
  { id: 3, bank: 'SBI',  amount: 300000 },
];
```

`map` — change every item. Returns a **new array of the same length**.

```
const names = loans.map(loan => loan.bank);   // ['SBI', 'HDFC', 'SBI']
```

This is what you use to turn data into JSX in React.

`filter` — keep only some items. Returns a **shorter array**.

```
const sbi = loans.filter(loan => loan.bank === 'SBI');   // 2 items
```

This is what you use for search and filter features.

`find` — get the **first matching item** itself, not an array.

```
const one = loans.find(loan => loan.id === 2);   // { id: 2, ... }
```

`reduce` — turn the whole array into a single value.

```
const total = loans.reduce((sum, loan) => sum + loan.amount, 0);
// 1600000
```

Read it as: start at 0, then add each loan's amount to the running total.

`forEach` — just loop. Returns nothing. Use it only for side effects.

Why this matters in React

`map`, `filter` and `reduce` do **not** change the original array. They return a new one. React needs a new array to notice that something changed. So these methods fit React perfectly, and a loop with `push` does not.

Q9. Event bubbling and event delegation

Bubbling

When you click a button inside a div, the click event fires on the button first, then travels **up** to the div, then to the body. This travelling up is bubbling.

```
<div onClick="...">        ← 3rd
  <ul onClick="...">       ← 2nd
    <li onClick="...">     ← 1st (you clicked here)
```

Delegation, and why it is useful

Imagine a list of 500 loan rows, each with a delete button.
You could add 500 click listeners. That is slow and uses memory.

Instead, add **one** listener on the parent, and check what was clicked:

```
list.addEventListener('click', (e) => {
  const btn = e.target.closest('button');
  if (!btn) return;                  // they clicked somewhere else
  deleteRow(btn.dataset.id);
});
```

One listener instead of 500. It also works for rows added later, because the parent was already listening.

Two methods to know

- `e.stopPropagation()` — stop the event from bubbling further up.
- `e.preventDefault()` — stop the browser's default action, like a form reloading the page or a link navigating away.

Q10. Debounce and throttle ● (practise writing this)

The problem

A user types "axis bank" in a search box. That is 9 characters. If you call the API on every keystroke, you send **9 requests**. Eight of them are wasted, and the results may arrive out of order.

Debounce: wait until the user stops

"Only run after there has been no typing for 400ms."
Use for: **search boxes**, autocomplete, saving a draft.

```
function debounce(fn, delay = 400) {
  let timer;                          // kept alive by a closure

  return function (...args) {
    clearTimeout(timer);              // cancel the previous plan
    timer = setTimeout(() => fn(...args), delay);   // make a new plan
  };
}
```

Every keystroke cancels the previous timer. Only the last one survives.
Result: 9 keystrokes, **1 request**.

Throttle: run at most once every X ms

"No matter how many times this fires, run it only once per 300ms."
Use for: **scroll**, resize, mouse move, infinite scroll.

```
function throttle(fn, limit = 300) {
  let waiting = false;

  return function (...args) {
    if (waiting) return;              // still in the cool-down period
    fn(...args);
    waiting = true;
    setTimeout(() => { waiting = false; }, limit);
  };
}
```

The difference in one line

Debounce waits for a pause. Throttle limits the rate.

Say this

> "Debounce runs the function only after the user stops for a set time, which is what you want for a search box. Throttle runs it at a fixed maximum rate, which is what you want for scroll events. Without debouncing, a nine letter search sends nine API calls instead of one."

Q11. Hoisting

Before running your code, JavaScript scans it and makes room for all the declarations. This is called hoisting.

```
sayHi();                    // <span class="tick">✓</span> works
function sayHi() { ... }    // function declarations are fully hoisted

console.log(a);             // undefined  ← var is hoisted but empty
var a = 5;

console.log(b);             // ❌ ReferenceError
let b = 5;                  // let/const are hoisted but locked
```

That locked period for `let` and `const` is called the **Temporal Dead Zone (TDZ)**.
It is a good thing. It gives you a clear error instead of a silent `undefined`.

Q12. Shallow copy vs deep copy ●

```
const original = { name: 'Sujith', address: { city: 'Hyderabad' } };

const shallow = { ...original };
shallow.name = 'Kumar';               // <span class="tick">✓</span> original.name is safe
shallow.address.city = 'Chennai';     // ❌ original.address.city ALSO changed

const deep = structuredClone(original);
deep.address.city = 'Chennai';        // <span class="tick">✓</span> original is safe
```

Spread copies only the top level. Anything nested is still the same object in memory. This is the reason behind a very common React bug: "I changed the state but the screen did not update", or "I changed one item and another item changed too".

Q13. localStorage vs sessionStorage vs cookies

	Size	How long it lasts	Sent to the server?
`localStorage`	about 5 MB	Forever, until cleared	No
`sessionStorage`	about 5 MB	Until the tab is closed	No
Cookie	about 4 KB	Until it expires	**Yes, on every request**

Which one for a login token?

An **httpOnly cookie** is the safest. `httpOnly` means JavaScript cannot read it. So if an attacker injects a script into your page, that script still cannot steal the token. Anything in `localStorage` can be read by any script on the page.

For a product handling student loan data, this is worth saying out loud.

⚠ **In Next.js:** `localStorage` does not exist on the server. Reading it in a Server Component or during the first render will crash. Read it inside `useEffect`, which only runs in the browser.

Q14. Quick questions they fire at the end

- **`slice` vs `splice`?** `slice` returns a copy and leaves the original alone. `splice` changes the original array.
- **`typeof null`?** `'object'`. This is a famous bug in JavaScript from 1995. It was never fixed because too much code depends on it.
- **How to check for an array?** `Array.isArray(x)`. `typeof []` gives `'object'`.
- **`NaN === NaN`?** `false`. NaN is not equal to anything, including itself. Use `Number.isNaN(x)`.
- **Higher order function?** A function that takes a function, or returns one. `map`, `filter` and `debounce` are all higher order functions.
- **Pure function?** Same input always gives the same output, and it changes nothing outside itself. Easy to test and easy to reason about.
- **Synchronous vs asynchronous?** Synchronous blocks the next line until it finishes. Asynchronous starts the work and lets the next line run.

✓ Check yourself before moving on

You should be able to do these **without looking**:

1. Explain a closure using the counter example, in about 30 seconds.
2. Explain why `1 4 3 2` is the answer in the event loop question.
3. Write the `debounce` function from memory.

4. Explain why `0 || 8.5` gives the wrong answer and `0 ?? 8.5` gives the right one.

02 · React (Most important technical file)

Time needed: 75 minutes

This is the main skill for the job. Expect 15 to 20 minutes of React questions, plus a coding task built on React.

First, the context: what problem does React solve?

Life before React

To change something on a page, you had to find the element and change it by hand:

```
document.getElementById('count').innerText = count;
document.getElementById('warning').style.display = count > 5 ? 'block' : 'none';
document.getElementById('btn').disabled = count === 0;
```

This is called **imperative** code. You give step by step instructions.

The problem is keeping everything in sync. If `count` changes in five places, you must remember to update all three lines in all five places. Miss one and the screen shows the wrong thing. In a big app this becomes impossible to manage.

What React does instead

You describe **what the screen should look like for a given state**. React works out what to change in the page.

```
function Counter({ count }) {
  return (
    <div>
      <p>{count}</p>
      {count > 5 && <p>That is a lot</p>}
      <button disabled={count === 0}>Reset</button>
    </div>
  );
}
```

You never touch the DOM. You change `count`, and React updates the screen.
This is called **declarative** code.

The second big idea: components

A component is a reusable piece of UI. Write `<LoanCard />` once, use it on the home page, the search page and the comparison page. This is what the job description means by "reusable components".

Say this

> "React is declarative. Instead of writing instructions to change the DOM, I describe what the UI should look like for the current state, and React updates the DOM for me. That removes a whole class of bugs where the screen and the data go out of sync. The component model also means I build something like a loan card once and reuse it everywhere."

Q1. What is the Virtual DOM? ●

What it means

The real DOM is the actual page in the browser. Changing it is **slow**, because the browser may have to recalculate layout and repaint pixels.

The Virtual DOM is a **plain JavaScript copy** of what the page should look like. It is just an object in memory, so making it is cheap and fast.

How an update works

1. State changes.
2. React builds a **new** Virtual DOM tree.
3. React compares it with the **previous** tree. This comparison is called **diffing**.
4. React finds the smallest set of real changes and applies only those.

Example: a list of 100 rows and you change one name. React does not rebuild 100 rows in the browser. It changes one piece of text.

The rules React uses when comparing

- Different element type (`<div>` became `<span>`)? Throw away the old one, build new.
- Same type? Keep the element, only update the attributes that changed.
- For lists, match items by their `key` .

A trick question: "Is the Virtual DOM faster than direct DOM manipulation?"

The honest answer scores better than "yes".

> "Not always. Hand written DOM code can be faster if it is done perfectly. The Virtual DOM gives you very good performance without you having to think about it, and it makes the code much easier to maintain. That trade is worth it."

Q2. Why do lists need a `key`? Why is index a bad key? ●

What a key does

A key is a **name tag** for each item in a list. It tells React "this is the same item as before" across renders.

```
{loans.map(loan => <LoanCard key={loan.id} loan={loan} />)}
```

Without keys, React can only go by position. If you insert an item at the top, React thinks every item changed, and does far more work than needed.

Why index is dangerous, with a concrete example

Say you have three text inputs:

```
index 0 → Apple    (user typed "hello" in this box)
index 1 → Banana
index 2 → Cherry
```

Now delete **Apple**. The array becomes `[Banana, Cherry]`.

With `key={index}`:

```
index 0 → Banana   ← React thinks: "item 0 is still here, its text just changed"
index 1 → Cherry
```

React **reuses** the first input box. The text "hello" the user typed is still sitting in it, but now it is next to Banana. The data moved but the DOM did not.

With `key={item.id}`:
React sees that the item with id "apple" is gone. It removes that input box completely, along with the typed text. Correct behaviour.

When is index acceptable?

Only when the list never changes order, never gets items removed, and has no internal state. A static footer menu is fine. Anything the user can edit is not.

Q3. Props vs State ●

The simple difference

- **Props** come from **outside** (the parent). The child cannot change them.
- **State** lives **inside** the component. The component can change it.

Think of a component as a function. Props are the arguments passed in. State is a variable it keeps for itself.

```jsx
function LoanCard({ bank, rate }) {          // props, given by the parent
  const [expanded, setExpanded] = useState(false);   // state, owned here

  return (
    <div>
      <h3>{bank} — {rate}%</h3>
      <button onClick={() => setExpanded(!expanded)}>
        {expanded ? 'Hide' : 'Show'} details
      </button>
    </div>
  );
}
```

How does a child send data back to the parent?

Data flows **down** through props. Events flow **up** through callbacks.
The parent passes a function down, and the child calls it.

```jsx
// Parent
<SearchBar onSearch={(text) => setQuery(text)} />

// Child
<input onChange={(e) => props.onSearch(e.target.value)} />
```

"Lifting state up"

If two sibling components need the same value, neither can own it. Move the state **up** to their closest shared parent, then pass it down to both.

```
       Parent  ← state lives here
      /      \
  SearchBar   ResultList
```

Q4. `useState` — the three traps ●

```jsx
const [count, setCount] = useState(0);
//     ↑ value   ↑ function to change it   ↑ starting value
```

Trap 1: state updates are not instant

```jsx
setCount(count + 1);
setCount(count + 1);
// count only goes up by 1, not 2
```

Both lines read the same old `count`. React batches the updates and applies them after the current work finishes.

Fix — use the function form. React passes you the latest value:

```
setCount(c => c + 1);
setCount(c => c + 1);
// now it goes up by 2 <span class="tick">✓</span>
```

Rule: if the new value depends on the old value, use the function form.

Trap 2: never change state directly

```
items.push(newItem);
setItems(items);          // ❌ screen does not update
```

React checks "is this the same object as before?" It is the same array, just with one more item inside. So React decides nothing changed and skips the re-render.

```
setItems([...items, newItem]);              // <span class="tick">✓</span> a brand new array
setUser({ ...user, verified: true });       // <span class="tick">✓</span> a brand new object
```

Always create a new array or object.

Trap 3: expensive starting values

```
useState(readFromLocalStorage())      // ❌ runs on every single render
useState(() => readFromLocalStorage())// <span class="tick">✓</span> runs once, on the first render
```

Q5. `useEffect` ●

What it is for

`useEffect` is for talking to things **outside** React. For example:

- fetching data from an API
- setting a timer
- adding a browser event listener
- saving to localStorage

The dependency array controls when it runs

```
useEffect(() => { ... });           // after EVERY render (rarely what you want)
useEffect(() => { ... }, []);       // once, when the component appears
useEffect(() => { ... }, [userId]); // when userId changes
```

Cleanup: the part people forget

The function you `return` from an effect is the cleanup. React runs it when the component disappears, and before running the effect again.

```
useEffect(() => {
  const id = setInterval(tick, 1000);

  return () => clearInterval(id);   // ← cleanup
}, []);
```

Without cleanup, the timer keeps running after the component is gone. It tries to update a component that no longer exists. That is a memory leak, and you will see warnings in the console.

Things that need cleanup: timers, event listeners, API subscriptions, and in flight fetch requests.

● When should you NOT use useEffect?

This question separates good candidates from average ones.

Do not use an effect for data you can calculate.

```
// ❌ Wrong: extra state, extra render, and it can go out of sync
const [filtered, setFiltered] = useState([]);
useEffect(() => {
  setFiltered(items.filter(i => i.bank === bank));
}, [items, bank]);

// <span class="tick">✓</span> Right: just calculate it while rendering
const filtered = items.filter(i => i.bank === bank);
```

The second version is shorter, has no extra render, and can never be stale.

Why does my effect run twice in development?

React 18 StrictMode mounts your component, unmounts it, and mounts it again on purpose. It does this to reveal missing cleanup. It only happens in development, never in production.

Q6. The hooks list

Hook	What it is for
`useState`	Store a value that changes and should re-render the screen
`useEffect`	Talk to the outside world (API, timers, browser)
`useContext`	Read shared data without passing props down every level
`useRef`	Store a value that survives renders but does **not** re-render
`useMemo`	Remember a calculated **value** so it is not recalculated
`useCallback`	Remember a **function** so it is not recreated
`useReducer`	Manage complex state with many different actions
Custom hook	Your own reusable `useSomething` function

🔴 The Rules of Hooks, and the reason behind them

Only call hooks at the top level. Never inside an `if`, a loop, or a nested function.

```
if (isLoggedIn) {
  const [name, setName] = useState('');   // ❌ never do this
}
```

Why? React does not know your hooks by name. It tracks them by **the order they are called**. First hook, second hook, third hook. If an `if` skips one, the order shifts, and React gives you the wrong value from the wrong hook.

Q7. `useMemo`, `useCallback` and `useRef` 🔴

`useMemo` — remember a value

```
const total = useMemo(() => {
  return loans.reduce((sum, l) => sum + l.amount, 0);
}, [loans]);
```

The calculation only runs again when `loans` changes. On other renders React reuses the stored answer.

`useCallback` — remember a function

```
const handleAdd = useCallback((id) => {
  setCart(c => [...c, id]);
}, []);
```

Why would a function need remembering? Every render creates a **new** function object. Even though the code is identical, it is a different object in memory. If you pass it to a child wrapped in `React.memo`, the child sees a "new" prop and re-renders anyway. `useCallback` keeps the same function object.

`useRef` — a box that does not trigger re-renders

Two uses.

1. Reach a DOM element:

```
const inputRef = useRef(null);
useEffect(() => { inputRef.current.focus(); }, []);
return <input ref={inputRef} />;
```

2. Keep a value between renders without re-rendering:

```
const timerId = useRef(null);        // storing a timer id
const isFirstRender = useRef(true);  // a flag
```

useState vs useRef in one line: changing state re-renders the screen. Changing a ref does not.

● "Should you wrap everything in useMemo?"

Say **no**. This is a maturity signal.

> "No. Memoization is not free. React still has to store the value and compare the dependencies on every render. If the calculation is cheap, memoizing it can be slower than just doing it. I profile with React DevTools first and memoize where there is a real problem."

Q8. What causes a re-render? How do you stop unnecessary ones? ●

A component re-renders when:

1. Its own state changed.
2. Its props changed.
3. **Its parent re-rendered.** This happens even if its props are identical.

4. A context it uses changed.

Point 3 surprises people. By default, when a parent re-renders, all its children re-render too.

How to fix it, in the order you should try

1. `React.memo` — skip the re-render if the props are the same.

```
const LoanCard = React.memo(function LoanCard({ loan }) { ... });
```

2. `useCallback` / `useMemo` **on the props you pass to it.**
`React.memo` compares props. If you pass a fresh function or object every render, the comparison always fails and `React.memo` does nothing.

3. Move state down. If only a small part of the page uses a piece of state, put that state in a smaller component. Then only that part re-renders.

4. Split your contexts. If one context holds both the theme and the cart, every cart change re-renders everything using the theme too.

Q9. Controlled vs uncontrolled inputs ●

Controlled — React state holds the value. React is the source of truth.

```
const [email, setEmail] = useState('');
<input value={email} onChange={e => setEmail(e.target.value)} />
```

Uncontrolled — the DOM holds the value. You read it when you need it.

```
const emailRef = useRef();
<input ref={emailRef} defaultValue="" />
// later: emailRef.current.value
```

Which one, and why

Use **controlled** by default. Because the value is in state, you can:

- validate as the user types
- disable the submit button when the form is invalid
- format the value, for example adding commas to an amount
- reset the form easily

Use **uncontrolled** for very large forms where a re-render on every keystroke is too slow, or for file inputs. `<input type="file">` is always uncontrolled, because for security reasons JavaScript cannot set its value.

Q10. Context API ●

The problem it solves: prop drilling

The logged in user is needed deep in the tree. Without context you pass it through every level, even components that do not use it:

```
App (has user)
 └ Layout      (passes user, does not use it)
    └ Sidebar  (passes user, does not use it)
       └ Menu  (passes user, does not use it)
          └ Avatar   ← finally uses it
```

How context fixes it

```jsx
const AuthContext = createContext(null);

// near the top
<AuthContext.Provider value={{ user, logout }}>
  <Layout />
</AuthContext.Provider>

// anywhere below, at any depth
const { user } = useContext(AuthContext);
```

Good for values that rarely change: the current user, the theme, the language.

● The limitation you must mention

Every component using a context re-renders whenever the context value changes.

And this is a common mistake:

```jsx
<AuthContext.Provider value={{ user, logout }}>
```

That object literal is **created fresh on every render**. So it is always "new", and every consumer re-renders every time, even if `user` did not change.

Fix:

```jsx
const value = useMemo(() => ({ user, logout }), [user]);
<AuthContext.Provider value={value}>
```

> "Context solves prop drilling, but it is not a state manager. It is not optimised for values that change often, because every consumer re-renders. For server data I would use React Query instead, and for large client state, Redux Toolkit or Zustand."

Q11. Custom hooks

A custom hook is just a function whose name starts with `use` and which calls other hooks. It lets you reuse **logic**, not UI.

```
function useDebounce(value, delay = 500) {
  const [debounced, setDebounced] = useState(value);

  useEffect(() => {
    const timer = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(timer);
  }, [value, delay]);

  return debounced;
}

// using it
const query = useDebounce(searchText, 400);
useEffect(() => { callApi(query); }, [query]);
```

Important detail: each component that calls `useDebounce` gets its **own separate state**. The hook shares the logic, not the data.

Have one custom hook ready to describe. It is a strong signal.

Q12. Code splitting and error boundaries

Code splitting with lazy and Suspense

By default, all your code is bundled into one big JavaScript file. The user downloads the admin dashboard even if they never open it.

```
const Dashboard = React.lazy(() => import('./Dashboard'));

<Suspense fallback={<Spinner />}>
  <Dashboard />
</Suspense>
```

Now the dashboard code downloads only when it is needed. The initial page loads faster. This connects directly to file 04 on performance.

Error boundary

If any component throws an error during render, React unmounts the whole app and the user sees a blank white page. An error boundary catches that and shows a fallback instead.

```
class ErrorBoundary extends React.Component {
  state = { hasError: false };
  static getDerivedStateFromError() { return { hasError: true }; }
  render() {
    if (this.state.hasError) return <p>Something went wrong.</p>;
    return this.props.children;
  }
}
```

This is the one place where you still write a class component. Hooks cannot do it yet. In practice most teams use the `react-error-boundary` package.

Q13. Class components vs function components

Class (old)	Function + hooks (current)
`componentDidMount`	`useEffect(fn, [])`
`componentDidUpdate`	`useEffect(fn, [dep])`
`componentWillUnmount`	the cleanup `return` inside `useEffect`
`this.state`, `this.setState`	`useState`

> "Function components with hooks are the standard now. There is less boilerplate, no confusion with `this`, and logic can be reused through custom hooks instead of higher order components. I can read class components in older code, but I write function components."

Q14. Which state management would you choose?

Answer with a decision path, not a favourite library:

1. **Local** `useState` first. Most state belongs to one component.
2. **Lift it up** if two siblings need it.

3. **Context** for values that are shared widely and change rarely, like the user or the theme.
4. **React Query or SWR** for server data. Most "global state" is really a copy of server data, and these libraries handle caching, refetching, loading and error states for you.
5. **Redux Toolkit or Zustand** only when client state is genuinely complex.

> "I would not add Redux to a project just to hold a user object. That is extra setup with no benefit."

Q15. Quick questions

- **What is JSX?** It looks like HTML but it is JavaScript. A build tool converts `<div>Hi</div>` into `React.createElement('div', null, 'Hi')`. The browser never sees JSX.
- **Why `className` and not `class`?** `class` is a reserved word in JavaScript.
- **What is a Fragment?** `<>...</>` groups elements without adding an extra `<div>` to the page.
- **How do you render conditionally?** `{isLoading && <Spinner />}` or a ternary.
- ⚠ **The `0` trap:** `{count && <p>...</p>}` renders a literal `0` on the screen when count is 0, because `0` is falsy but React still prints it. Write `{count > 0 && <p>...</p>}`.
- **`React.memo` vs `useMemo`?** `React.memo` wraps a component. `useMemo` caches a value inside a component.
- **Can you read `key` as a prop?** No. React uses it internally, the component never receives it.

✓ Check yourself before moving on

1. Explain the four things that cause a re-render, and three ways to reduce them.
2. Write `useDebounce` from memory.
3. Explain the "index as key" problem using the delete example, in 30 seconds.
4. Explain why `items.push(x); setItems(items)` does not update the screen.

03 · Next.js (Your best chance to stand out)

Time needed: 75 minutes

The job description names Next.js. Their business depends on Google search. Most candidates at this level can use React but cannot explain **why Next.js exists**.
If you can, you move to the top of the list.

First, the context: how a normal React app loads

When you build a plain React app (Create React App or Vite), the server sends the browser an almost empty file:

```
<html>
  <body>
    <div id="root"></div>              ← completely empty
    <script src="/bundle.js"></script> ← 300 KB of JavaScript
  </body>
</html>
```

Then, in the browser:

1. Download the HTML. It has no content.
2. Download the JavaScript bundle.
3. Parse and run the JavaScript.
4. React builds the page.
5. React asks the API for data.
6. Finally the user sees the loan offers.

This is called **Client Side Rendering (CSR)**. The client (browser) does the work.

Two problems with this

Problem 1: the user waits. On a mid range Android phone on 4G, steps 2 to 6 can take several seconds. The user stares at a blank white screen.

Problem 2: Google sees an empty page. When Google's crawler first visits, the HTML has no content in it. Google can run JavaScript, but it does so later, in a second pass. That makes indexing slower and less reliable. Other crawlers, like LinkedIn and WhatsApp link previews, are much worse at it.

For WeMakeScholars this second problem is serious. Students find them by searching Google for education loans. If the pages do not rank, there are no students.

Q1. Why Next.js instead of plain React? ●●

What Next.js changes

Next.js runs React **on the server first**. The server builds the finished HTML and sends that to the browser.

```
<html>
  <body>
    <h1>Education Loans for Study Abroad</h1>   ← real content, immediately
    <ul><li>SBI – 9.15%</li><li>HDFC – 9.55%</li></ul>
    <script src="/bundle.js"></script>
  </body>
</html>
```

The user sees content straight away. Google sees content straight away.

What else you get for free

- **File based routing.** Create a file, get a URL. No router setup.
- **Automatic code splitting.** Each page loads only its own JavaScript.
- `next/image` **.** Automatic image optimisation.
- **Built in SEO metadata.** Titles and descriptions per page.
- **API routes.** Small backend endpoints inside the same project.

Say this

> "A plain React app sends an empty HTML file and then builds the page in the browser. That means the user waits with a blank screen, and a crawler's first look at the page has no content in it. Next.js renders on the server, so the browser gets real HTML immediately. That helps both first paint speed and SEO. It also gives me file based routing, code splitting and image optimisation without extra setup. For a site like WeMakeScholars, where students arrive from Google, that is not optional, it is how the pages get found."

If they push back: "But Google runs JavaScript now"

> "It does, but rendering happens in a second pass that is queued, so indexing is slower and less reliable. Other crawlers such as Bing and social preview bots are worse at it. Server rendered HTML removes the risk. And it still helps real users on slow phones, which is the bigger benefit anyway."

Q2. CSR, SSR, SSG, ISR ●● (the most valuable answer in this guide)

These are four different **times** at which the HTML can be built.

1. CSR — Client Side Rendering

Built in the browser, after JavaScript loads.
The plain React behaviour described above.
Good for: pages behind a login, where SEO does not matter. A student's dashboard.

2. SSR — Server Side Rendering

Built on the server, fresh for every single request.
A student opens the page → the server runs the code → sends finished HTML.

Good for: pages that are different for each user, or must be perfectly up to date.
Cost: the server does work on every request, so it is the slowest of the server options and puts the most load on your infrastructure.

```
// App Router
const res = await fetch(url, { cache: 'no-store' });   // no cache = SSR
```

3. SSG — Static Site Generation

Built once, at build time. When you run `npm run build`, Next.js creates the HTML files. They sit on a CDN and are served instantly.

Good for: content that is the same for everyone and rarely changes. An "About us" page, a blog post, a landing page.
Cost: to change the content you must rebuild and redeploy the whole site.

```
const res = await fetch(url);     // cached by default = SSG
```

4. ISR — Incremental Static Regeneration

The best of both. Built like SSG, but Next.js rebuilds the page quietly in the background every N seconds.

```
const res = await fetch(url, { next: { revalidate: 3600 } });   // every hour
```

How it works:

- A visitor gets the stored HTML instantly, like SSG.
- If the page is older than one hour, Next.js rebuilds it in the background.
- The next visitor gets the fresh version.
- Nobody waits for the rebuild.

Good for: content that is the same for everyone but changes now and then.

Summary table

	When HTML is made	Speed for user	Fresh data?	Use for
CSR	In the browser	Slowest first paint	Yes	Logged in dashboards
SSR	On every request	Good	Always fresh	Personalised pages
SSG	Once, at build	Fastest	Only at build time	Blog, about page
ISR	At build + refreshed on a timer	Fastest	Fresh within N seconds	Listings, prices

● The follow up question: "Which would you use for their bank listing page?"

This is where you win the interview. Think out loud:

> "The list of partner banks and their interest rates is the **same for every visitor**, so SSR would be wasteful. The server would rebuild an identical page for every student who lands on it.
>
> But the rates **do change** sometimes, and rebuilding the whole site for one rate change is not practical.
>
> So I would use **ISR with a revalidate of about an hour**. Visitors get a static file served from the CDN, which is the fastest possible, and the content is never more than an hour old.
>
> The student's own application status page is different. That is personal to them and must be current, so that would be SSR or client side behind a login."

Practise saying that out loud tonight. It is your single strongest answer.

Q3. App Router vs Pages Router ●

Next.js has two routing systems. Version 13 introduced the new one.

	Pages Router (older)	App Router (newer)
Folder	`/pages`	`/app`
A page	`pages/about.js`	`app/about/page.js`
Get data	`getServerSideProps`, `getStaticProps`	`async` component with `await fetch`
Default component type	Client	**Server Component**
Shared layout	`_app.js`	`layout.js`, and it can be nested
Loading state	You build it	`loading.js` file
Error state	`_error.js`	`error.js` per folder

Be honest about which one you have used. Then show you understand the other.

> "I have built mostly with [X]. I know the App Router is the direction Next.js is going, and the main differences are Server Components by default and data fetching directly in the component instead of `getServerSideProps`."

Q4. Server Components vs Client Components ●●

This is the newest concept, and the one people find confusing. Here is the simple version.

The idea

In the App Router, **components run on the server by default**. Their JavaScript is never sent to the browser at all. Only the HTML they produced is sent.

If a component needs to be interactive (a button, an input, anything with `useState` or `onClick`), you mark it with `'use client'` at the top of the file. That component's code is then sent to the browser.

Example

```
// app/loans/page.js  — Server Component (no 'use client')
export default async function LoansPage() {
  // You can await directly. This runs on the server.
  const res = await fetch('https://api.example.com/loans', {
    next: { revalidate: 60 }
  });
  const loans = await res.json();

  return (
    <div>
      <h1>Education Loans</h1>
      <SearchBar />              {/* interactive, so it is a client component */}
      <LoanList loans={loans} /> {/* just displays data, stays on server */}
    </div>
  );
}
```

```jsx
// components/SearchBar.jsx
'use client';                    // ← this file goes to the browser

import { useState } from 'react';

export default function SearchBar() {
  const [query, setQuery] = useState('');
  return <input value={query} onChange={e => setQuery(e.target.value)} />;
}
```

What each can do

Server Component	Client Component
Sends **zero JavaScript** to the browser	Adds to the JavaScript bundle
Can `await` data directly	Cannot `await` at the top level
Can read secrets and hit a database safely	Runs in the browser, so no secrets
❌ No `useState`, `useEffect`, `onClick`	✓ All hooks and events work

● The rule to state

> "Keep components on the server by default, and push `'use client'` down to the leaves, meaning the small interactive pieces. If I put `'use client'` at the top of the page, every component inside it also goes to the browser and I have thrown away the benefit."

Environment variables (a good detail to know)

In Next.js, environment variables are **server only** by default. That is safe.
Only variables starting with `NEXT_PUBLIC_` are sent to the browser.

```
DATABASE_PASSWORD=secret        ← safe, server only
NEXT_PUBLIC_API_URL=https://... ← visible to anyone in the browser
```

Never put a secret in a `NEXT_PUBLIC_` variable. Anyone can read it in DevTools.

Q5. Routing

The folder structure **is** the routing.

```
app/
  layout.js                → wraps everything (nav, footer)
  page.js                  → /
  about/page.js            → /about
  loans/page.js            → /loans
  loans/[id]/page.js       → /loans/123      (dynamic)
  blog/[...slug]/page.js   → /blog/a/b/c     (catch all)
  loading.js               → shown while the page loads
  error.js                 → shown if the page throws
  not-found.js             → the 404 page
  api/leads/route.js       → an API endpoint at /api/leads
```

Navigation

```
import Link from 'next/link';
<Link href="/loans">View loans</Link>
```

● Why `<Link>` and not `<a>`?

- `<a href="/loans">` makes the browser throw the whole page away and download everything again from scratch.
- `<Link>` fetches only the new page's data and swaps it in. Much faster.
- `<Link>` also **prefetches** the page when the link scrolls into view, so the navigation feels instant.
- Importantly, `<Link>` still renders a real `<a>` tag underneath. So Google can still follow it and middle click still works.

Programmatic navigation

```
'use client';
import { useRouter } from 'next/navigation';   // App Router

const router = useRouter();
router.push('/dashboard');
```

⚠ App Router uses `next/navigation`. Pages Router uses `next/router`. Mixing them up is a common mistake, so mention that you know the difference.

Q6. `next/image` ●

```
import Image from 'next/image';

<Image
  src="/bank-logo.png"
  alt="SBI bank logo"
  width={400}
  height={300}
  priority        // only for the main image at the top of the page
/>
```

What it does automatically

1. **Converts the format.** Serves WebP or AVIF to browsers that support them. These are much smaller than JPEG or PNG.
2. **Resizes.** Generates several sizes and serves the right one for the device. A phone does not download a 2000 pixel wide image.
3. **Lazy loads.** Images below the fold only download when the user scrolls near them.
4. **Reserves space.** Because you gave width and height, the browser leaves an empty box of the right size. The page does not jump when the image arrives. This directly improves your CLS score (see file 04).

`priority` turns **off** lazy loading for your main hero image, so it loads immediately instead of waiting.

The same idea for fonts

`next/font` downloads Google Fonts at build time and serves them from your own server. No extra connection to Google, and no flash of invisible text.

Q7. SEO in Next.js ● (their highlighted requirement)

```
// A fixed title, in app/layout.js or any page.js
export const metadata = {
  title: 'Education Loans for Study Abroad | WeMakeScholars',
  description: 'Compare education loan offers from 15+ banks and NBFCs. Free.',
  openGraph: { title: '...', images: ['/og-image.png'] },
  alternates: { canonical: 'https://example.com/loans' },
};
```

```
// A title built from data, for a dynamic page
export async function generateMetadata({ params }) {
  const bank = await getBank(params.id);
  return {
    title: `${bank.name} Education Loan Interest Rates 2026`,
    description: `Current ${bank.name} education loan rates, eligibility and documents.`,
  };
}
```

Also worth naming:

- `app/sitemap.js` and `app/robots.js` — Next.js generates these files for you.
- **JSON-LD structured data** — extra machine readable information in a `<script type="application/ld+json">` tag. It can give you rich results in Google, like an FAQ dropdown under your link. A loan FAQ page is a perfect fit for this.

Q8. What is hydration?

The idea

The server sends finished HTML, so the user sees the page immediately. But that HTML is static. Buttons do not work yet.

Then the JavaScript arrives and React attaches all the event handlers to the existing HTML. That process is called **hydration**. Like adding water to something dried.

There is a short window where the page **looks** ready but is not yet interactive.

Hydration errors

If the HTML the server made does not match what React builds in the browser, you get a hydration mismatch warning.

Common causes:

```
<p>{new Date().toLocaleTimeString()}</p>   // server time ≠ browser time
<p>{Math.random()}</p>                     // different every time
<p>{window.innerWidth}</p>                 // window does not exist on the server
```

Fix: move it into `useEffect`, which only runs in the browser, or load the component with `next/dynamic` and `{ ssr: false }`.

Q9. API Routes

You can put small backend endpoints inside your Next.js project.

```
// app/api/leads/route.js
export async function POST(request) {
  const body = await request.json();

  // The API key stays on the server. The browser never sees it.
  await fetch('https://crm.internal/leads', {
    method: 'POST',
    headers: { Authorization: `Bearer ${process.env.CRM_SECRET}` },
    body: JSON.stringify(body),
  });

  return Response.json({ ok: true }, { status: 201 });
}
```

Useful when you need to hide an API key, reshape a response, or handle a form submission without building a separate backend service.

Q10. Quick questions

- `next/dynamic`**?** Load a component only when needed. With `{ ssr: false }` it skips the server entirely, which you need for libraries that use `window`, like charts and maps.
- **Middleware?** Code that runs before a request is completed. Used for auth redirects and A/B tests.
- **Streaming?** With `loading.js` or `Suspense`, Next.js sends the page shell first and streams in the slow parts as they become ready. The user sees something sooner.
- **How do you check what rendering a page uses?** Run `next build`. It prints a symbol next to each route showing static, SSG or dynamic.
- **Deployment?** Vercel is the simplest since they build Next.js. It also runs on any Node server with `next build && next start`.

✓ Check yourself before moving on

1. Say the CSR / SSR / SSG / ISR table out loud, then pick one for a bank listing page **and give the reason**.
2. Explain Server vs Client Components, and the "push `'use client'` to the leaves" rule.
3. Explain hydration in three sentences.

> **If your Next.js experience is only from tutorials, say so.** Then show the understanding above. Interviewers forgive thin experience. They do not forgive bluffing that falls apart under one follow up question.

04 · Performance and SEO

Time needed: 60 minutes

In their job description, the line **"Making Things Fast"** was highlighted in their own document. That means the person who wrote it cares about it. Treat this as a named requirement, not general knowledge.

First, the context: why speed is a business problem here

WeMakeScholars gets students from Google search. So two things follow.

1. Speed affects ranking. Google uses page speed as a ranking signal. A slow page appears lower in results, so fewer students find it.

2. Speed affects conversion. Most students are on mid range Android phones on 4G. If the page takes 5 seconds, many leave before it loads. The loan form never gets filled.

So on this product, performance work is **lead generation**, not polish. Say that once in the interview. It reframes you from someone who writes code to someone who understands why the code matters.

Q1. Core Web Vitals ●●

Google measures three things. Learn all three with their numbers.

LCP — Largest Contentful Paint

"When does the main content appear?"
It measures the time until the biggest thing on screen is visible. Usually the hero image or the main heading.

Good: under 2.5 seconds.

Common causes of a bad score:

- A huge unoptimised hero image
- Slow server response
- CSS or JavaScript blocking the page from rendering

Fixes: optimise and preload the hero image, use SSR or SSG so HTML arrives ready, use a CDN, compress files.

CLS — Cumulative Layout Shift

"Does the page jump around while loading?"

You have felt this. You go to tap a button, an image finishes loading above it, everything shifts down, and you tap the wrong thing.

Good: under 0.1.

Common causes:

- Images without width and height, so no space is reserved
- Banners or ads inserted at the top after load
- A custom font loading late and changing the text size

Fixes: always set width and height on images (or use `next/image`), reserve space for anything that loads late, use `font-display: swap`.

INP — Interaction to Next Paint

"When I tap something, how fast does the page respond?"
This replaced FID in 2024.

Good: under 200 milliseconds.

Cause: JavaScript is busy doing something long, so the browser cannot respond to the tap. Remember from file 01 that JavaScript is single threaded.

Fixes: break long tasks into smaller ones, debounce expensive handlers, reduce unnecessary re-renders, ship less JavaScript.

The one line summary

LCP = is it there. CLS = does it stay still. INP = does it respond.

Two more terms you may hear: **TTFB** (Time to First Byte, how fast your server replies, good is under 800ms) and **FCP** (First Contentful Paint, the first pixel of any content).

Q2. "The page is slow. How do you find out why?" ●

Never answer with a list of fixes. They are testing your **process**.

Step 1: Measure

- **Lighthouse** in Chrome DevTools for a score and a list of problems.
- **Network tab** to see what is being downloaded and how big it is.
- **Performance tab** to record and find long JavaScript tasks.
- Throttle to **Slow 4G** and 4x CPU slowdown, so you see what a real phone sees.

Say the line: *"I do not optimise what I have not measured."*

Step 2: Decide which category the problem is in

Category	Symptom in DevTools
Network	Very large files, or too many requests
Server	Long wait before the first byte arrives (TTFB)
Rendering	CSS or JS blocking, blank screen for a long time
JavaScript	Long tasks in the Performance tab, page feels frozen

Step 3: Fix the biggest item first

Usually images, then bundle size. Do not start with micro optimisations.

Step 4: Measure again

Prove the change worked. Compare the before and after Lighthouse numbers.

Q3. The optimisation toolbox

Learn these in **groups**. Reciting a random list sounds memorised. Grouping sounds like understanding.

Group 1: Images (usually the biggest win)

- **Modern formats.** WebP and AVIF are much smaller than JPEG. `next/image` does this automatically.
- **Right size.** Do not send a 2000 pixel image into a 400 pixel space.
- **Lazy load** anything below the fold, so it only downloads when scrolled to.
- **Set width and height** so the layout does not jump. This fixes CLS.

Group 2: JavaScript

- **Code splitting.** Split by route with `next/dynamic` or `React.lazy`, so the user only downloads the page they are on.
- **Tree shaking.** Import only what you use. `import _ from 'lodash'` pulls in the whole library. `import debounce from 'lodash/debounce'` pulls in one function.
- **Check your dependencies.** Is that date library 70 KB when you only format one date? `date-fns` is smaller than `moment`.
- **Delay third party scripts.** Analytics and chat widgets are often the slowest things on a page. Load them last with `next/script` and `strategy="lazyOnload"`.

Group 3: Rendering

- Use **SSG or ISR** so the HTML is ready before JavaScript loads.

- Reduce re-renders with `React.memo` and `useMemo`, but only where you measured a problem.
- **Virtualise long lists.** With `react-window`, a 5,000 row table only puts about 20 rows in the DOM at a time. The rest are not rendered until you scroll.
- **Debounce** search inputs and **throttle** scroll handlers.

Group 4: Network and delivery

- Use a **CDN** so files are served from a server near the user.
- Enable **gzip or Brotli** compression.
- **Self host fonts** with `next/font` so there is no extra connection.
- **Avoid waterfalls.** Two API calls that do not depend on each other should run with `Promise.all`, not one after the other.

Q4. Technical SEO for a React or Next app ●

The core issue

If content only appears after JavaScript runs, crawlers may not see it reliably. So anything that must rank should be **server rendered**.

The checklist

- **Unique title and meta description on every page.** Use `generateMetadata` in Next.
- **Semantic HTML.** One `<h1>` per page, then `<h2>` and `<h3>` in order. Use `<nav>`, `<main>`, `<article>`. Crawlers use this structure to understand the page.
- `alt` **text on every meaningful image.** Helps accessibility and image search.
- **Canonical URL** so Google knows which version of a page is the real one, when the same content is reachable from two URLs.
- `sitemap.xml` **and** `robots.txt`**.** Next.js can generate both from files.
- **Structured data (JSON-LD).** Machine readable information about the page. An FAQ page can get an expandable FAQ shown directly in Google results. A loan FAQ is a perfect example, and worth volunteering in the interview.
- **Open Graph tags** so shared links show a proper preview card.
- **Real** `<a>` **links for internal navigation.** A `<div onClick={router.push}>` is invisible to a crawler. `<Link>` renders a real anchor, so it is fine.
- **Mobile first and fast.** Google indexes the mobile version of your site.

Q5. Accessibility, briefly

Worth a minute. It overlaps with SEO because both rely on good structure.

- Use real elements. A `<button>` is focusable, works with the Enter key, and is announced correctly by screen readers. A `<div onClick>` does none of that.
- Connect labels to inputs with `htmlFor` and `id`.
- Keep visible focus outlines. Do not remove them with `outline: none`.
- Text contrast of at least 4.5 to 1.
- Use `aria-label` only when there is no visible text, such as an icon only button. ARIA is a patch. Correct HTML is better.
- Quick test: press Tab through the page. Can you reach and use everything?

Q6. Caching, in simple terms

Type	Where	Example
Browser cache	User's device	JS and CSS files, cached for a year
CDN cache	Edge servers worldwide	Static pages served near the user
Data cache	Your app	Next.js `revalidate`, React Query
Memoization	Inside React	`useMemo`, `React.memo`

Useful detail: build tools add a hash to filenames like `main.a3f9c2.js`. So you can cache that file forever. When the code changes, the filename changes, and the browser downloads the new one automatically.

Q7. Quick questions

- **How do you reduce the initial bundle size?** Split code by route, lazy load below the fold components, remove heavy libraries, and check with `@next/bundle-analyzer`.
- **What is lazy loading?** Delaying the download of something until it is actually needed.
- **What is a CDN?** Servers spread around the world that hold copies of your files, so users download from a nearby one instead of a distant origin server.
- **`defer` vs `async` on a script tag?** Both download in the background. `async` runs as soon as it arrives, in no guaranteed order. `defer` runs after the HTML is parsed, in order. Use `defer` for anything that touches the page.
- **Reflow vs repaint?** Reflow means the browser recalculates positions and sizes, which is expensive. Repaint means it redraws colours, which is cheaper. This is why you animate `transform` and `opacity` and never `width` or `top`. Those two skip layout entirely and run on the GPU.

✓ Check yourself before moving on

1. Name LCP, CLS and INP with their target numbers and one fix each.
2. Give the four step process for "the page is slow".
3. Explain in one sentence why client side rendering is a risk for SEO.

05 · HTML, CSS and Responsive Design

Time needed: 30 minutes tonight, quick review tomorrow

The first line of the job description is about "translating UI/UX wireframes into clean, performant, responsive code". This topic shows up more in the coding round than in questions, but a few questions are still likely.

Q1. What is semantic HTML and why does it matter? ●

Semantic means the tag describes what the content **is**, not how it looks.

```
<!-- Not semantic: everything is a div -->
<div class="header">
  <div class="nav">
    <div class="btn" onclick="submit()">Apply</div>
  </div>
</div>

<!-- Semantic -->
<header>
  <nav>
    <button onclick="submit()">Apply</button>
  </nav>
</header>
```

Three reasons it matters

1. **Accessibility.** A screen reader user can jump straight to `<nav>` or `<main>`. A `<button>` can be reached with the Tab key and pressed with Enter. A `<div onClick>` cannot do either, unless you add a lot of extra code.
2. **SEO.** Crawlers use the structure to understand what the page is about.
3. **Readability.** The next developer can see the shape of the page instantly.

Tags worth knowing: `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<aside>`, `<footer>`, `<figure>`, `<button>`, `<form>` and `<label>`.

Q2. The box model ●

Every element is a box made of four layers, from inside out:

```
+---------------- margin ----------------+   space outside
|  +------------- border -------------+  |
|  |  +---------- padding ---------+  |  |   space inside
|  |  |        content             |  |  |
```

The problem, and the one line fix

By default, `width: 300px` means the **content** is 300px. Add 20px of padding and a 2px border, and the box actually takes 344px on screen. This breaks layouts.

```
* { box-sizing: border-box; }
```

Now `width: 300px` means the whole box is 300px, including padding and border. This is what everyone expects, which is why this line is in almost every CSS reset.

Q3. Flexbox vs Grid — when do you use which? ●

Flexbox works in one direction. A row, or a column.
Grid works in two directions. Rows and columns together.

Use Flexbox for

Navbars, a row of buttons, centring something, spacing items apart, the inside of a card.

```
.navbar {
  display: flex;
  justify-content: space-between;  /* spread along the main axis */
  align-items: center;             /* centre on the cross axis */
  gap: 1rem;
}
```

Use Grid for

Page layouts, card galleries, dashboards, anything with real rows and columns.

```
.cards {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(260px, 1fr));
  gap: 1.5rem;
}
```

● That grid line is worth explaining, it sounds senior

Read it as: *"Fit as many columns as you can. Each must be at least 260px wide. Any leftover space is shared equally between them."*

The result is a card grid that goes 4 across on a laptop, 2 on a tablet and 1 on a phone, **with no media queries at all**. Demo that line and explain it.

They work together

A common real layout is Grid for the page, Flexbox inside each card.

Centring, the short version

```
.parent { display: grid; place-items: center; }
```

Q4. Responsive design and mobile first ●

What mobile first means

Write the styles for the smallest screen first. Then use `min-width` media queries to **add** complexity for bigger screens.

```
/* base = mobile */
.card { padding: 1rem; }

@media (min-width: 768px)  { .card { padding: 2rem; } }   /* tablet and up */
@media (min-width: 1024px) { .card { padding: 3rem; } }   /* desktop and up */
```

Why not the other way round?

If you write desktop styles first and use `max-width` to shrink them, the phone downloads all the desktop rules and then overrides them. The weakest device does the most work. Mobile first gives the simplest styles to the simplest device.

The essentials

- **The viewport meta tag.** Without this line, nothing responsive works at all:

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

- **Relative units** instead of fixed pixels: `rem`, `%`, `vw`.
- **Fluid sizing** with `clamp()`:

```
font-size: clamp(1rem, 2.5vw, 2rem);   /* min, preferred, max */
```

- `max-width: 100%` **on images** so they never overflow their container.

Q5. Position values

Value	What it does
`static`	Default. Sits in normal flow.
`relative`	Moves relative to itself, but keeps its original space.
`absolute`	Removed from flow. Positioned against the nearest positioned parent.
`fixed`	Positioned against the viewport. Stays put when scrolling.
`sticky`	Normal until it reaches a threshold, then behaves like fixed.

`sticky` is what you use for a table header that stays visible while scrolling. It needs a `top` value to work.

Q6. Specificity, in simple terms

When two rules target the same element, the more specific one wins.

```
inline style   = 1000
#id            = 100
.class         = 10
element (div)  = 1
```

`!important` beats everything. Treat it as a warning sign.

> "If I need `!important`, it usually means my selectors have got too specific somewhere. I would rather fix the selector than fight it."

Q7. Quick questions

- **`display: none` vs `visibility: hidden` vs `opacity: 0`?**
  Removed from the layout / hidden but still takes up space / fully visible to the layout and **still clickable**.
- **Pseudo class vs pseudo element?** `:hover` is a state. `::before` creates a new sub element.
- **`rem` vs `em`?** `rem` is relative to the root font size, which is predictable. `em` is relative to the parent, so it multiplies when nested.
- **CSS variables?** Declare `--brand: #14356b` on `:root`, then use it anywhere as `background: var(--brand)`. They can be changed at runtime with JavaScript, which is how dark mode is usually built.

- **Why is my `z-index` not working?** Two common reasons. It only applies to positioned elements. And a parent with `transform` or `opacity` less than 1 creates a new stacking context that traps its children.
- **What do you think of Tailwind?** Have an opinion but do not be extreme.

  > "Utility classes keep the styling next to the markup, and unused CSS is removed at build time so the file stays small. The downside is that the JSX gets noisy. I pull repeated patterns into components."

- **CSS Modules?** `styles.card` generates a unique class name, so styles cannot clash between files. Next.js supports them by default.

✓ Check yourself before moving on

1. Flexbox vs Grid, decided in one sentence.
2. Write the `auto-fit` and `minmax` grid line from memory and explain it.
3. Explain mobile first and why `min-width` is better than `max-width`.

06 · Connecting to REST APIs

Time needed: 30 minutes

The job description says: *"Working closely with our backend team to plug REST APIs and data into the frontend smoothly."* Expect this in the coding round.

First, the context: what an API call actually is

Your React app has no data of its own. The data lives on a server. An API is the agreed way to ask for it.

```
Browser                          Server
   |                               |
   |  GET /api/loans?bank=SBI      |
   | ----------------------------> |
   |                               |  looks in the database
   |  200 OK  [ {...}, {...} ]     |
   | <---------------------------- |
   |                               |
render the list
```

REST is just a set of conventions for how those requests should look. Use nouns for the address (`/loans`), and a verb to say what you want to do.

Q1. The HTTP basics they may check

Verb	Meaning
`GET`	Read data. Should never change anything.
`POST`	Create something new.
`PUT`	Replace an entire item.
`PATCH`	Update part of an item.
`DELETE`	Remove an item.

Status codes

Code	Meaning
200	OK
201	Created

Code	Meaning
400	Bad request. You sent something wrong.
401	**Not logged in.** "Who are you?"
403	**Logged in, but not allowed.** "I know who you are, and no."
404	Not found
429	Too many requests, you are rate limited
500	Server error. Not your fault.

- The 401 vs 403 difference is a common quick question. Learn the two sentences.

Q2. `fetch` vs `axios`

	`fetch`	`axios`
Setup	Built into the browser	A library, about 13 KB
Error on 404 or 500	**No. You must check yourself.**	Yes, it throws automatically
JSON	You call `res.json()`	Automatic both ways
Timeout	Not built in	Built in option
Interceptors	No	Yes

What is an interceptor? A function that runs on every request or every response.
It lets you write something once instead of in fifty places.

```
// attach the token to every request, in one place
axios.interceptors.request.use(config => {
  config.headers.Authorization = `Bearer ${getToken()}`;
  return config;
});

// handle every expired session, in one place
axios.interceptors.response.use(
  res => res,
  err => {
    if (err.response?.status === 401) redirectToLogin();
    return Promise.reject(err);
  }
);
```

> "For a small project `fetch` is enough. On a bigger codebase I prefer axios for the interceptors, so the auth header and the 401 handling live in one place instead of being repeated in every call."

Q3. The four states ●●

This is the most important idea in this file.

Every screen that loads data has four possible states:

1. **Loading** — the request is in flight. Show a spinner or a skeleton.
2. **Error** — something failed. Show a message and a retry button.
3. **Empty** — it worked, but there is nothing to show. Show "No loans match your filters."
4. **Success** — show the data.

Most candidates only handle loading and success. Saying all four out loud is a genuine hiring signal, because empty and error are what users actually hit.

The full pattern

```
function LoanList() {
  const [loans, setLoans]     = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError]     = useState(null);

  useEffect(() => {
    const controller = new AbortController();

    async function load() {
      try {
        setLoading(true);
        setError(null);

        const res = await fetch('/api/loans', { signal: controller.signal });

        // fetch does NOT throw on 404 or 500, so check it yourself
        if (!res.ok) throw new Error(`Request failed: ${res.status}`);

        setLoans(await res.json());
      } catch (err) {
        // ignore the error we caused ourselves by cancelling
        if (err.name !== 'AbortError') setError(err.message);
      } finally {
        setLoading(false);           // always runs, success or failure
      }
    }

    load();

    return () => controller.abort();  // cleanup: cancel if we unmount
  }, []);

  if (loading)       return <Spinner />;
  if (error)         return <ErrorState message={error} onRetry={load} />;
  if (!loans.length) return <p>No loans match your filters.</p>;

  return <ul>{loans.map(l => <LoanCard key={l.id} loan={l} />)}</ul>;
}
```

Point out these three things without being asked

1. `res.ok` because `fetch` does not throw on a bad status.
2. `AbortController` **in the cleanup** for two reasons, explained next.
3. `finally` so loading always stops, even when the request fails.

Q4. Race conditions, and why `AbortController` matters ●

The problem, with a real example

The user types in a search box.

```
t=0ms    types "ax"    → request A starts
t=200ms  types "axis"  → request B starts
t=600ms  request B finishes → screen shows results for "axis"  <span class="tick">✓</span>
t=900ms  request A finishes → screen shows results for "ax"    ❌ WRONG
```

The older, slower request finished last and overwrote the correct results. The search box now shows results that do not match what is typed.

The fixes

1. **Cancel the old request** with `AbortController`. This is the standard fix.
2. **Debounce** so fewer requests are made in the first place.
3. Use **React Query**, which handles this for you.

`AbortController` in the cleanup solves two problems at once: it prevents this race condition, and it stops React warning that you updated state on a component that no longer exists.

Q5. CORS — you will be asked at least what it is

What it is

CORS is a **browser** security rule. A page loaded from `site-a.com` is not allowed to read a response from `site-b.com`, unless `site-b.com` explicitly permits it with a response header:

```
Access-Control-Allow-Origin: https://site-a.com
```

The two things to say

1. **It is enforced by the browser, not the server.** This is why the exact same request works fine in Postman but fails in the browser.
2. **It is fixed on the backend, not the frontend.** The server must send the header. A dev proxy is a local workaround, not a real fix.

For requests that are not simple, the browser first sends an `OPTIONS` request to ask permission. That is called a **preflight**.

Q6. React Query and SWR — worth mentioning

> "A lot of what people call global state is really just a cached copy of server data. React Query handles that properly: caching, background refetching, deduplicating identical requests, retries, and loading and error flags. It replaces most of the `useEffect` plus `useState` code above, including the race condition handling."

```
const { data, isLoading, error } = useQuery({
  queryKey: ['loans'],
  queryFn: getLoans,
});
```

Q7. Authentication on the frontend

- Store the token in an **httpOnly cookie** rather than `localStorage`. `httpOnly` means JavaScript cannot read it, so an injected script cannot steal it.
- Attach it with an interceptor so it is written once.
- Handle **401** in one place: clear the session and send the user to login.
- **Never put secrets in frontend code.** In Next.js, anything named `NEXT_PUBLIC_*` is visible in the browser. If you need to use a secret key, call it from an API route on the server instead.

Q8. "How do you work with the backend team?" — the job description's real question

Answer as a teammate, not just a coder:

> "I ask for the contract early: the endpoint, the request shape, the response shape and the error shape. Agreeing on that before either side builds saves a lot of rework.
>
> If the API is not ready, I mock the response so I am not blocked, and swap the URL in later.
>
> I check pagination and the error format up front, because those are usually what break during integration.
>
> And if a response shape does not fit the UI well, I would rather raise it than quietly write conversion code in five different components."

✓ Check yourself before moving on

1. Name the four states, in order.
2. Explain the race condition example, and how `AbortController` fixes it.
3. Explain CORS in two sentences, including whose job it is to fix.
4. What does `fetch` do on a 404?

07 · Machine Coding Round

Time needed: 60 minutes. And you must TYPE these, not read them.

Reading code makes you feel prepared. It does not make you prepared. Open CodeSandbox or StackBlitz and build problems 1 and 2 from an empty file. That is how you should spend this hour.

First, the context: what they are actually grading

They are not checking whether your code is beautiful. In 45 minutes nobody writes beautiful code. They are checking six things:

1. Can you break the problem into state and components?
2. Do you know React well enough to not fight it?
3. Do you handle the cases that are not the happy path?
4. Do you talk while you work?
5. Do you notice your own mistakes?
6. Would it be pleasant to sit next to you for a day?

How to run the round

1. Clarify for 60 seconds. Do not start typing immediately.

> "Should the filtering happen on the client, or should I call the API each time?"
> "Roughly how many items are we dealing with?"
> "Should the state survive a refresh?"

Candidates who ask questions score higher than candidates who start typing. Every time.

2. Say your plan out loud before you write it.

> "I will keep the typed text in state, debounce it, then filter the list while rendering. Then I will handle the empty case."

3. Build the ugly working version first. Get it working, then improve it. Do not style anything until the logic works.

4. Keep talking while you type. Silence makes the interviewer think you are stuck. Narrate what you are doing, even if it feels awkward.

5. Say the edge cases out loud, even the ones you do not have time to build. *"I would add an error state here if I had more time"* still earns you the point.

6. Refactor at the end if there is time. Pull something into a custom hook, or split a component. It shows you know what good code looks like.

Common problems at this level: search and filter a list, todo CRUD, a counter, an accordion or tabs, a star rating, form validation, fetching and displaying users, pagination, a modal, a countdown timer.

Problem 1 ● — Search and filter a list, with debounce

The most commonly asked machine coding question in Indian frontend interviews.

What they are testing: derived state, debouncing, keys, and whether you remember the empty state.

```jsx
import { useState, useEffect, useMemo } from 'react';

function useDebounce(value, delay = 400) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const t = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(t);
  }, [value, delay]);
  return debounced;
}

export default function BankSearch({ banks }) {
  const [query, setQuery] = useState('');
  const [type, setType]   = useState('all');
  const debouncedQuery    = useDebounce(query);

  // derived state — NOT useState + useEffect
  const filtered = useMemo(() => {
    return banks.filter(b => {
      const matchesQuery = b.name.toLowerCase().includes(debouncedQuery.toLowerCase().trim());
      const matchesType  = type === 'all' || b.type === type;
      return matchesQuery && matchesType;
    });
  }, [banks, debouncedQuery, type]);

  return (
    <div>
      <input
        value={query}
        onChange={e => setQuery(e.target.value)}
        placeholder="Search banks…"
        aria-label="Search banks"
      />
      <select value={type} onChange={e => setType(e.target.value)}>
        <option value="all">All</option>
        <option value="public">Public</option>
        <option value="private">Private</option>
        <option value="nbfc">NBFC</option>
      </select>

      {filtered.length === 0
        ? <p>No banks match “{debouncedQuery}”.</p>
        : <ul>{filtered.map(b => <li key={b.id}>{b.name} — {b.rate}%</li>)}</ul>}
    </div>
  );
}
```

Explain these four things while you type

1. The filtered list is calculated during render, not stored in state.

> "I could have put `filtered` in state and updated it in a `useEffect`, but then I have two sources of truth that can go out of sync, plus an extra render. It is derived data, so I calculate it."

2. The debounce reduces work.

> "Without this, typing nine characters runs the filter nine times. If it were an API call, that would be nine requests instead of one."

3. The key is a stable id, never the array index.

4. The edge cases are handled. `.trim()` removes accidental spaces, and `.toLowerCase()` on both sides makes the search case insensitive. And when nothing matches, the user sees a message instead of a blank area.

Problem 2 ● — Fetch + loading / error / empty / retry

```jsx
import { useState, useEffect, useCallback } from 'react';

export default function Users() {
  const [users, setUsers]     = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError]     = useState(null);

  const load = useCallback(async (signal) => {
    try {
      setLoading(true);
      setError(null);
      const res = await fetch('https://jsonplaceholder.typicode.com/users', { signal });
      if (!res.ok) throw new Error(`HTTP ${res.status}`);
      setUsers(await res.json());
    } catch (e) {
      if (e.name !== 'AbortError') setError(e.message);
    } finally {
      setLoading(false);
    }
  }, []);

  useEffect(() => {
    const c = new AbortController();
    load(c.signal);
    return () => c.abort();
  }, [load]);

  if (loading) return <p>Loading…</p>;
  if (error)   return (
    <div role="alert">
      <p>Couldn’t load users: {error}</p>
      <button onClick={() => load()}>Retry</button>
    </div>
  );
  if (!users.length) return <p>No users found.</p>;

  return <ul>{users.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}
```

Why this scores well

- All four states are handled: loading, error, empty, and success.
- **The retry button.** Almost nobody adds this. An error state the user cannot escape from is a dead end.
- `res.ok` is checked, because `fetch` does not throw on a 404 or 500.

- The request is cancelled on unmount with `AbortController`.
- `finally` means loading always stops, even when the request fails.

Problem 3 — Todo list (CRUD + no mutation)

```jsx
export default function Todos() {
  const [todos, setTodos] = useState([]);
  const [text, setText]   = useState('');

  const add = (e) => {
    e.preventDefault();
    const value = text.trim();
    if (!value) return;                                  // edge case: empty input
    setTodos(t => [...t, { id: crypto.randomUUID(), text: value, done: false }]);
    setText('');
  };

  const toggle = (id) =>
    setTodos(t => t.map(todo => todo.id === id ? { ...todo, done: !todo.done } : todo));

  const remove = (id) => setTodos(t => t.filter(todo => todo.id !== id));

  return (
    <>
      <form onSubmit={add}>
        <input value={text} onChange={e => setText(e.target.value)} />
        <button type="submit">Add</button>
      </form>
      <ul>
        {todos.map(todo => (
          <li key={todo.id}>
            <input type="checkbox" checked={todo.done} onChange={() => toggle(todo.id)} />
            <span style={{ textDecoration: todo.done ? 'line-through' : 'none' }}>{todo.text}</span>
            <button onClick={() => remove(todo.id)}>×</button>
          </li>
        ))}
      </ul>
      <p>{todos.filter(t => !t.done).length} remaining</p>
    </>
  );
}
```

Details worth pointing out

- `<form onSubmit>` **instead of a button** `onClick`**.** This means the Enter key works, which is what users expect. Interviewers notice this.

- **Functional updaters** (`setTodos(t => ...)`) so you always work from the latest state.
- `map` **and** `filter` **never change the original array.** They return a new one, which is exactly what React needs to detect a change.
- `crypto.randomUUID()` **for the id**, not the array index.
- **The remaining count is calculated, not stored** in another piece of state.

Problem 4 — Form with validation

```
export default function LeadForm() {
  const [values, setValues]   = useState({ name: '', email: '', phone: '' });
  const [errors, setErrors]   = useState({});
  const [touched, setTouched] = useState({});
  const [status, setStatus]   = useState('idle');   // idle | submitting | success | error

  const validate = (v) => {
    const e = {};
    if (!v.name.trim()) e.name = 'Name is required';
    if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(v.email)) e.email = 'Enter a valid email';
    if (!/^[6-9]\d{9}$/.test(v.phone)) e.phone = 'Enter a valid 10-digit mobile number';
    return e;
  };

  const handleChange = (e) => {
    const next = { ...values, [e.target.name]: e.target.value };
    setValues(next);
    if (touched[e.target.name]) setErrors(validate(next));   // re-validate only after blur
  };

  const handleBlur = (e) => {
    setTouched(t => ({ ...t, [e.target.name]: true }));
    setErrors(validate(values));
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    const found = validate(values);
    setErrors(found);
    setTouched({ name: true, email: true, phone: true });
    if (Object.keys(found).length) return;
    setStatus('submitting');
    try {
      await fetch('/api/leads', { method: 'POST', body: JSON.stringify(values) });
      setStatus('success');
    } catch { setStatus('error'); }
  };

  return (
    <form onSubmit={handleSubmit} noValidate>
      {['name', 'email', 'phone'].map(field => (
        <div key={field}>
          <label htmlFor={field}>{field}</label>
          <input id={field} name={field} value={values[field]}
                 onChange={handleChange} onBlur={handleBlur}
                 aria-invalid={!!errors[field]} />
          {touched[field] && errors[field] && <small role="alert">{errors[field]}</small>}
```

```
        </div>
      ))}
      <button disabled={status === 'submitting'}>
        {status === 'submitting' ? 'Submitting…' : 'Submit'}
      </button>
      {status === 'success' && <p>Thanks — we'll call you shortly.</p>}
    </form>
  );
}
```

The three details that make this look experienced

1. Errors only appear after the user leaves a field (`onBlur`).
Showing "Enter a valid email" while someone is still typing the first letter of their email is bad UX. That is what the `touched` state is for.

2. The submit button is disabled while the request is in flight.
Otherwise an impatient user clicks three times and creates three applications.

3. One `handleChange` for all fields, using `e.target.name`. Writing a separate handler for every field does not scale.

Problem 5 — Accordion (controlled, one open at a time)

```
function Accordion({ items }) {
  const [openId, setOpenId] = useState(null);
  return (
    <div>
      {items.map(item => {
        const isOpen = openId === item.id;
        return (
          <div key={item.id}>
            <button aria-expanded={isOpen}
                    onClick={() => setOpenId(isOpen ? null : item.id)}>
              {item.title} {isOpen ? '−' : '+'}
            </button>
            {isOpen && <p>{item.body}</p>}
          </div>
        );
      })}
    </div>
  );
}
```

Why one `openId` instead of a boolean per item

If each item had its own `isOpen` boolean, you would have to remember to close all the others every time one opens. It is easy to get wrong. With a single `openId`,

"only one open at a time" is guaranteed by the shape of the state itself.

If they ask for multiple open at once, hold a `Set` of ids instead.

`aria-expanded` tells a screen reader whether the section is open. And if this is an FAQ, you can mention that adding FAQ structured data would make it eligible for a rich result in Google.

Problem 6 — Star rating

```
function Rating({ value = 0, onChange, max = 5 }) {
  const [hover, setHover] = useState(0);
  return (
    <div onMouseLeave={() => setHover(0)}>
      {Array.from({ length: max }, (_, i) => i + 1).map(star => (
        <button key={star} type="button"
                aria-label={`Rate ${star} of ${max}`}
                onMouseEnter={() => setHover(star)}
                onClick={() => onChange(star)}>
          {star <= (hover || value) ? '★' : '☆'}
        </button>
      ))}
    </div>
  );
}
```

The trick is `hover || value`

While the mouse is over star 4, `hover` is 4, so stars 1 to 4 are filled as a preview. When the mouse leaves, `hover` becomes 0, which is falsy, so it falls back to the actual saved `value`. One line handles both the preview and the real state.

Problem 7 — Pagination (client-side)

```
function Paginated({ items, perPage = 10 }) {
  const [page, setPage] = useState(1);
  const totalPages = Math.ceil(items.length / perPage);
  const slice = items.slice((page - 1) * perPage, page * perPage);

  useEffect(() => { setPage(1); }, [items]);   // reset when the data set changes

  return (
    <>
      <ul>{slice.map(i => <li key={i.id}>{i.name}</li>)}</ul>
      <button disabled={page === 1} onClick={() => setPage(p => p - 1)}>Prev</button>
      <span>Page {page} of {totalPages || 1}</span>
      <button disabled={page >= totalPages} onClick={() => setPage(p => p + 1)}>Next</button>
    </>
  );
}
```

Edge cases to name out loud

- Disable Previous on page 1 and Next on the last page.
- An empty list should not show "Page 1 of 0".
- **Reset to page 1 when the data changes.** If the user is on page 5 and applies a filter that leaves 3 results, they would see an empty page. This is a very common bug.

(You have built `xpagination` before. Mention it.)

Problem 8 — Counter / timer (the `useRef` + cleanup check)

```
function Timer() {
  const [seconds, setSeconds] = useState(0);
  const [running, setRunning] = useState(false);
  const idRef = useRef(null);

  useEffect(() => {
    if (!running) return;
    idRef.current = setInterval(() => setSeconds(s => s + 1), 1000);   // functional updater!
    return () => clearInterval(idRef.current);                          // cleanup!
  }, [running]);

  return (
    <>
      <h1>{seconds}s</h1>
      <button onClick={() => setRunning(r => !r)}>{running ? 'Pause' : 'Start'}</button>
      <button onClick={() => { setRunning(false); setSeconds(0); }}>Reset</button>
    </>
  );
}
```

They are checking exactly two things here

1. `setSeconds(s => s + 1)` and not `setSeconds(seconds + 1)`.
The interval callback was created once. It captured `seconds` as 0 in a closure and will never see a newer value. So `seconds + 1` would be `0 + 1` forever. The function form always receives the latest value.

2. The `clearInterval` cleanup.
Without it, the timer keeps running after the component is gone and tries to update state that no longer exists.

Point both out while you type. They are the entire reason this problem is asked.

If they ask for plain JS/DOM instead

```
document.querySelector('#list').addEventListener('click', (e) => {   // delegation
  const btn = e.target.closest('button[data-id]');
  if (!btn) return;
  removeItem(btn.dataset.id);
});
```

✓ Tonight's task

Build **Problem 1** and **Problem 2** from an empty file, with no copy and paste.

If you can build those two while explaining what you are doing, you can handle anything they are likely to ask at this level.

08 · Frontend System Design

Time needed: 45 minutes

This section was missing from the first version of the guide. It matters, because at this level the system design round is where they check if you can **plan** before you code, not just type React.

First, the context: what "frontend system design" actually means

When a backend engineer hears "system design", they think about databases, servers and load balancing. **You will not be asked that.**

A frontend system design question sounds like this:

- "How would you build the loan search page?"
- "Design an autocomplete search box."
- "How would you structure a multi step application form?"
- "We have a page with 5,000 rows. How do you make it fast?"

They are checking six things:

1. Do you **ask questions** before you start?
2. Can you break a screen into **components**?
3. Do you know **where the state should live**?
4. Do you think about the **API**, not just the UI?
5. Do you handle **loading, error and empty** cases?
6. Do you think about **performance** without being asked?

You do not need a perfect answer. You need a clear, ordered way of thinking.

The framework: answer every design question in this order

Memorise these six steps. Use them for any question they ask.

```
1. CLARIFY     Ask 2-3 questions. Never start immediately.
2. COMPONENTS  Break the screen into a component tree.
3. STATE       Decide what state exists and who owns it.
4. DATA        What API calls, when, and in what shape.
5. EDGE CASES  Loading, error, empty, slow network, long text.
6. PERFORMANCE One or two improvements, with a reason.
```

Step 1 is the one candidates skip, and it is the one that scores most.
Starting to code before asking anything is the most common failure in this round.

Useful clarifying questions that work for almost any problem:

- "Is the filtering done on the client, or does the API do it?"
- "Roughly how many items are we showing? Ten, or ten thousand?"
- "Does this need to work for logged out users, and does it need to rank on Google?"
- "Is the data the same for everyone, or is it per user?"

That last question is powerful in a Next.js interview, because the answer decides SSG, ISR or SSR.

Design 1 • — An autocomplete search box

The most commonly asked frontend design question in India.

Step 1: Clarify

- "Does the API return results, or do I filter a list I already have?"
- "Should recent searches be remembered?"
- "Do we need keyboard navigation with arrow keys?"

Step 2: Components

```
SearchBox
 ├── <input>            the text field
 ├── SuggestionList     shown only when there are results
 │    └── SuggestionItem
 └── StatusArea         spinner / "no results" / error
```

Step 3: State

State	Where it lives	Why
`query`	SearchBox	what the user typed
`debouncedQuery`	SearchBox (custom hook)	what we actually search with
`results`	SearchBox	what came back
`loading`, `error`	SearchBox	to show the right status
`activeIndex`	SearchBox	which suggestion is highlighted

Step 4: Data flow

```
user types  →  query updates on every keystroke
            →  debounce waits 400ms of silence
            →  ONE API call fires
            →  results render
```

Step 5: Edge cases — say all of these out loud

- **Empty query.** Do not call the API at all. Close the dropdown.
- **No results.** Show "No banks found for axis". Do not show an empty box.
- **Error.** Show a short message and let them try again.
- **Race condition.** The user types "ax", then "axis". Two requests are running. If "ax" answers last, the screen shows the wrong list. Fix it by cancelling the old request with `AbortController`.
- **Clicking outside** should close the dropdown.
- **Keyboard.** Arrow keys move, Enter selects, Escape closes.

Step 6: Performance

- **Debounce** turns 9 keystrokes into 1 request. That is a 90% reduction in load on the backend.
- **Cache** results per query in a simple object, so pressing backspace does not refetch something you already have.
- If the list is very long, **virtualise** it so only visible rows are in the DOM.

The 60 second version to say out loud

> "I would hold the typed text in state, and pass it through a `useDebounce` hook so the API is only called after the user pauses for around 400 milliseconds. The request result goes into `results`, with separate `loading` and `error` state.
>
> I would show four states: loading, error, empty and results, because an empty dropdown with no message looks broken.
>
> The main bug to guard against is a race condition. If two requests are in flight, the slower older one can overwrite the newer results. I cancel the previous request with an `AbortController` in the effect cleanup. And I would cache results per query so backspacing does not refetch."

Design 2 ● — A loan listing page with filters and pagination

This is essentially their real product, so it is very likely to come up.

Step 1: Clarify

- "How many loans are there in total? Hundreds or thousands?"
- "Should filters appear in the URL, so a student can share the link?"
- "Does this page need to rank on Google?"

That last question changes the whole design, so ask it.

Step 2: Components

```
LoansPage
 ├── FilterPanel
 │    ├── BankTypeFilter (public / private / NBFC)
 │    ├── AmountRangeFilter
 │    └── ClearAllButton
 ├── ResultsHeader     "24 loans found"  + sort dropdown
 ├── LoanList
 │    └── LoanCard × N
 └── Pagination
```

Step 3: State, and the important decision

```
filters   = { type: 'public', maxRate: 10, page: 2 }
loans     = []
loading / error
```

The key insight to mention: keep the filters in the URL.

```
/loans?type=public&maxRate=10&page=2
```

Why this matters:

- The student can **share or bookmark** their filtered view.
- The **back button works** correctly.
- Refreshing the page keeps the filters.
- And for Next.js, the server can read those filters and render the page on the server, so it is crawlable.

If you keep filters only in `useState`, all four of those break. Saying this shows product thinking, not just React knowledge.

Step 4: Data — client side or server side filtering?

	Filter on client	Filter on server
Good when	Under ~500 items	Thousands of items

	Filter on client	Filter on server
Speed	Instant, no network	One request per change
Data sent	All items at once	Only the current page

> "With a few hundred loans I would fetch once and filter in memory, which feels instant. With thousands, I would send the filters to the API and paginate on the server, because downloading everything would be slow and wasteful on mobile data."

Step 5: Edge cases

- No results after filtering → "No loans match your filters" **plus a Clear filters button**. Do not leave a dead end.
- Changing a filter must **reset the page back to 1**. A very common bug.
- Disable Previous on page 1 and Next on the last page.
- Long bank names must not break the card layout.

Step 6: Performance and rendering strategy

> "The loan list is the same for every visitor, so I would use **ISR** with a revalidate of about an hour. That gives static file speed from the CDN, and the rates are never more than an hour stale. Images go through `next/image`. If the list ever grows into thousands of rows on one page, I would virtualise it."

Design 3 — A multi step application form

You have built one, so use your own project as the example.

Step 1: Clarify

- "How many steps, and can the user go back?"
- "If they close the browser, should the progress be saved?"
- "Is validation per step, or all at the end?"

Step 2: Components

```
FormWizard              ← owns ALL the form data
 ├── ProgressBar        step 2 of 4
 ├── StepPersonal
 ├── StepEducation
 ├── StepLoanAmount
 ├── StepReview
 └── NavButtons         Back / Next / Submit
```

Step 3: State — the important decision

All form data lives in the parent `FormWizard`, not in each step.

```
const [formData, setFormData] = useState({ name: '', email: '', course: '' });
const [step, setStep] = useState(1);
const [errors, setErrors] = useState({});
```

Why? If each step kept its own state, going back to step 1 would unmount step 2 and destroy everything typed there. Keeping the data in the parent means the steps become simple display components, and nothing is lost when moving between them.

(This is exactly the bug story from file 10. It is a real thing you fixed.)

Step 4 and 5: Validation and edge cases

- Validate the current step before allowing Next.
- Show errors only **after** the user leaves a field, not while they are typing the first letter.
- Disable Submit while the request is in flight, so it cannot be double submitted.
- Save progress to `localStorage` so a refresh does not lose everything.
- On submit failure, keep all the data and show a retry. Never clear the form.

Step 6: Performance

> "The steps that are not visible do not need to be rendered at all. If a step imported something heavy, like a document upload widget, I would lazy load it with `next/dynamic` so it is not in the initial bundle."

Design 4 — "How would you make a reusable component?"

They may hand you a button or a card and ask how you would design its API.

The principles, in simple words

1. It should not know where it is used.
A `LoanCard` should not contain `if (page === 'home')`. Pass differences in as props.

2. Props describe *what*, not *how*.

```
<Button variant="primary" size="sm" loading={isSaving}>Apply</Button>
```

Not `<Button backgroundColor="#0055ff" paddingLeft="12px">`. If you pass raw styles, every screen looks slightly different and you have no design system.

3. Use `children` for the content.

```
<Card>
  <h3>SBI</h3>
  <p>9.15%</p>
</Card>
```

This is far more flexible than `<Card title="SBI" subtitle="9.15%" />`, because the next screen will always need something you did not predict.

4. Pass the rest of the props through.

```
function Button({ variant, children, ...rest }) {
  return <button className={styles[variant]} {...rest}>{children}</button>;
}
```

Now `onClick`, `disabled`, `type` and `aria-label` all work without you adding them one by one.

5. Handle its own states. A button should support normal, hover, disabled and loading. A card should survive a very long title.

The warning sign to mention

> "If a component has grown to fifteen boolean props like `isHomePage`, `isCompact`, `hideFooter`, that usually means it is really two different components. I would split it rather than keep adding flags."

Design 5 — How would you structure a large Next.js project?

```
app/                    routes only
  layout.js
  page.js
  loans/
    page.js
    [id]/page.js
  api/
components/
  ui/                   generic, reusable: Button, Card, Input, Modal
  loans/                feature specific: LoanCard, FilterPanel
  layout/               Header, Footer, Sidebar
hooks/                  useDebounce, useFetch, useLocalStorage
lib/                    api client, formatters, constants
styles/
public/                 images, fonts, icons
```

The one rule worth stating:

> "I separate **generic UI components** from **feature components**. Anything in `components/ui` knows nothing about loans and could be copied into another project. Anything in `components/loans` is allowed to know about our data. That boundary stops the shared components from slowly filling up with business logic."

Design 6 — "This page is slow. How do you fix it?"

Never jump to a solution. Show a process. (Full detail is in file 04.)

```
1. MEASURE   Lighthouse + DevTools Network and Performance tabs.
             Throttle to Slow 4G. "I do not optimise what I have not measured."

2. CATEGORISE  Which is it?
   - Too many / too large downloads   → images, bundle
   - Slow server response (TTFB)      → caching, rendering strategy
   - Heavy JavaScript blocking        → long tasks, re-renders

3. FIX THE BIGGEST ONE FIRST   Usually images, then bundle size.

4. MEASURE AGAIN   Prove the change worked.
```

Then name concrete fixes: `next/image` for format and size, code splitting per route, removing heavy libraries, virtualising long lists, debouncing inputs, and moving pages to SSG or ISR so the HTML is ready before JavaScript loads.

Design 7 — How do you handle authentication on the frontend?

The simple flow

```
1. User submits email + password
2. Server checks them, returns a token
3. Token is stored          ← the important decision
4. Every later request sends the token
5. Server returns 401 when it expires → send user back to login
```

The decision to explain

> "I would keep the token in an **httpOnly cookie** rather than `localStorage`. `httpOnly` means JavaScript cannot read it, so if someone manages to inject a script into the page, they still cannot steal the session. Anything in `localStorage` is readable by any script running on the page. For a product handling student financial data, that difference matters."

The frontend pieces

- An **axios interceptor** to attach the token to every request, in one place.
- A second interceptor to catch **401** responses and redirect to login, in one place.
- **Protected routes.** In Next.js, middleware can redirect before the page even renders.
- An `AuthContext` so any component can read the current user.

Design 8 — A toast / notification system

A nice small design question, and it shows you understand context.

The problem

Any component anywhere might need to say "Saved successfully". You cannot pass a callback down to all of them.

The design

```
ToastProvider (context, sits near the root)
 ├── holds:  toasts = [{ id, message, type }]
 ├── gives:  showToast(message, type)
 └── renders: <ToastContainer> fixed in a corner
```

```
// any component, at any depth
const { showToast } = useToast();
showToast('Application submitted', 'success');
```

Edge cases to name: auto dismiss after a few seconds, allow manual close, stack multiple toasts instead of overwriting, and clear the timer on unmount so it does not fire after the component is gone.

The three sentences that work in any design answer

Keep these ready. They fit almost every question:

1. **"Before I start, can I ask two things about the requirements?"**
2. **"Every data driven screen has four states: loading, error, empty and success. Empty is the one people forget."**
3. **"Since this is a search driven business, I would check whether this page needs to be crawlable, because that decides whether it is client rendered or server rendered."**

✓ Check yourself before moving on

1. Say the six step framework from memory: clarify, components, state, data, edge cases, performance.
2. Give the autocomplete answer out loud in about 60 seconds.
3. Explain why filters should live in the URL, giving three reasons.
4. Explain client side vs server side filtering, and when you would use each.

09 · HR, Behavioural & the Offer Conversation

Budget: 45 minutes — and say every answer OUT LOUD.
Reading these silently is worth about 20% of saying them. Your mouth needs the reps, not your eyes.

---

1. ● "Tell me about yourself" — the first 90 seconds

This is the only answer worth memorising almost word-for-word, because it sets the tone for everything after. Structure: **Present → Path → Why here.**

> "I'm a frontend developer working primarily with React and JavaScript. Most of my work has been building interfaces from designs and wiring them to REST APIs — I've built a multi-step form with validation, a paginated data table, an expense tracker, and several component-level UI builds, all on my GitHub.
>
> What I've focused on recently is the part beyond 'it works' — how fast the page loads, how it behaves on a mid-range phone, and writing components other people can reuse.
>
> That's why this role stood out. WeMakeScholars gets students through search, so the frontend has to be fast and crawlable — that's exactly the Next.js and performance work I want to be doing full-time, and on a product where the page loading properly actually decides whether a student gets a loan."

Rules: under 2 minutes · no life story from school onwards · end on *why them*.
Practise it twice out loud tonight and twice tomorrow morning. That's it.

---

2. ● "Why WeMakeScholars?" — do the homework, it's 3 sentences

Use their own facts back at them:

> "Three reasons. It's a 10-year-old, 250-person organisation, so it's stable but still building — I'd rather own real features than maintain someone else's legacy code. Second, the product is free for students and funded under Digital India, so the frontend genuinely decides whether someone finds and completes a loan application — the work has a visible outcome. Third, it's a search-driven product, which means Next.js, SSR and Core Web Vitals aren't optional here. That's the skill set I want to grow in."

Never say: "I need a job", "I want to learn", "the package is good".
Do mention you know they were inaugurated by Dr. Shashi Tharoor / recognised by Forbes / won the ET award — once, lightly. It shows you read.

3. ● The filter questions — answer instantly and without hedging

These decide whether you move forward. Hesitation here reads as "will leave in six months", which is precisely what the 18-month clause exists to prevent.

"We need an 18-month commitment. Are you comfortable?"

> "Yes. I'm looking for a place to build depth, not to switch in a year — 18 months is about the minimum time to actually own something end to end. I'd just like to understand how it's documented in the offer."

(Asking that follow-up is not a red flag — it's professional. Do read the clause before you sign: ask whether it's a written commitment or a bond with a financial penalty, and whether any amount is recoverable if you leave early. Ask it calmly at offer stage, not in round one.)

"What's your notice period?" → State it plainly and give a joining date. "I can join in X days." *(Their own 2-month notice applies once you're employed there — nothing to answer now.)*

"Are you okay with work from office in Hyderabad?" → "Yes." Full stop. Don't open a negotiation you weren't asked to have.

"1st and 3rd Saturdays are working." → "That's fine — I saw it in the JD, and it makes sense given you work with banks."

"Timings are 10–11 AM login, 8 hours." → "Works for me."

> ⚠ If any of these are genuinely a problem for you, say so **now**, politely. Accepting and renegotiating later burns the relationship and your reference.

---

4. ● Salary — the CTC is 4–6 LPA, so play the band

"What are your salary expectations?"

If you have prior experience / a current CTC:

> "I'm looking in the range of ₹X to ₹Y. That said, I'm more focused on the role and the learning here than on a specific number — if the fit is right on both sides, I'm confident we can agree on something."

If you're early-career with no strong anchor:

> "I'd like to understand the band for this role first. I saw the JD mentions 4 to 6 LPA — I'm comfortable within that range, and I'd expect to be placed based on how the technical rounds go."

Tactics:

- **Don't name a number below 4** — you'd be bidding against yourself under their own published floor.
- If you want the top of the band, **earn it in the technical round first**, then cite it: "Based on the Next.js and performance work we discussed, I'd be looking at the upper end."
- Ask **CTC vs in-hand** breakdown at offer stage — how much is fixed, variable, and what's deducted. A 6 LPA CTC and a 6 LPA fixed are different jobs.
- Never lie about your current CTC. Payslip verification is routine.

---

5. Behavioural questions — answer with STAR

Situation → **T**ask → **A**ction → **R**esult. Keep it under 90 seconds and always land on a result.

"Tell me about a challenging bug."

> "In my multi-step form, state was resetting when users moved between steps. (S/T) I'd been remounting the step components, so their local state was being destroyed. (A) I lifted the form state into the parent and passed values down, so each step became a controlled presentational component. (R) The bug went away and the code got simpler — adding a new step became a config change instead of new state logic. It taught me to ask *who owns this state* before writing it."

"A time you disagreed with someone."

> "A design had a filter panel that pushed the content far down on mobile. I flagged that it would hurt the mobile experience, but instead of just objecting I built a quick version with a collapsible drawer so we could compare. We went with the drawer. I've learned that showing an alternative works far better than arguing about one."

"Tight deadline?" → Talk about scoping: shipping the core flow properly and flagging what you cut, rather than shipping everything half-done. Managers hire people who communicate trade-offs early.

"Your weakness?" → Real, but bounded, plus the fix:

> "I used to over-polish UI details before the feature was fully working. I've started building the working version first and reviewing the polish at the end — it keeps me from spending an hour on a hover state that gets redesigned anyway."
> *(Never "I'm a perfectionist" or "I work too hard". Both read as rehearsed.)*

"Where do you see yourself in 3 years?" → "Owning a significant part of a frontend codebase — the person who's the reference point for performance and

component architecture on the team." *(Aligns with 18 months. Don't say "starting my own company" or "moving abroad".)*

"How do you work with designers?" → "I go through the design early and ask about the states the mock doesn't show — loading, error, empty, long text, small screens. Catching those in a 10-minute conversation is far cheaper than in QA."

"How do you keep code maintainable?" → "Small components with one job, clear naming, no magic numbers, shared logic in custom hooks, and a folder structure someone new can navigate. My test is whether a teammate can change it without asking me how it works."

---

6. ● Questions YOU ask them — *never say "no, I'm good"*

Have 4 ready; ask 2–3. Ending with no questions reads as no interest.

Technical (ask these in the tech round):

1. "Is the frontend on the App Router or Pages Router, and are you migrating?"
2. "How do you currently handle rendering strategy for the public pages — mostly SSG/ISR, or SSR?"
3. "Do you track Core Web Vitals in production, and is there a performance budget?"
4. "What does the frontend-to-backend workflow look like — do you get API contracts up front?"
5. "Is there a shared component library, or does each page build its own?"

Role / team (ask in the manager or HR round):

6. "What would a successful first three months in this role look like?"
7. "How big is the frontend team, and would I own features end to end?"
8. "Is there code review, and who reviews frontend work?"

The one that makes you memorable — ask it in the manager round:

> "What's the biggest frontend problem you're hoping this hire helps solve?"

Then **listen**, and connect your answer to it. That's how you close.

---

7. Logistics for tomorrow

- Laptop charged + charger · backup mobile hotspot · water · pen and notebook.
- 2 printed copies of your resume, plus a copy open on your laptop.
- Your GitHub open in a tab — and **your best 2 projects running**, so "can you show me?" gets a yes in five seconds, not a five-minute build.
- Reach 15 minutes early. If it's online, join 5 minutes early and test camera/mic.
- Dress: smart casual — collared shirt. Slightly overdressed beats underdressed.

- Know your own resume cold. **Anything on it is fair game** — if you listed a technology you touched once, be ready to say "I've used it at a project level."

---

8. If you get stuck in the room

- **Don't freeze, narrate.** "Let me think about that for a second."
- **Don't bluff.** "I haven't used that directly. My understanding is X — is that the direction you mean?" That answer has never cost anyone an offer. Bluffing has.
- **Ask for the goal.** "Is the tricky part here the state, or the API side?"
- If you fumble a question, let it go. Candidates lose interviews by mentally re-litigating question 4 while answering question 7.

10 · Pitching Your Projects

Budget: 45 minutes — out loud, standing up, twice.

> ⚠ **Verify this file against your actual resume.** Your resume PDF is on your local machine and this session can't read it, so the project list below comes from your **public GitHub repositories**. The interviewer will be reading the PDF, so where the two disagree, the PDF wins — edit this file to match.

Why this matters more than any single technical question

At 4–6 LPA, everyone in the pipeline has a todo app. **The differentiator isn't the project — it's whether you can explain a decision you made inside it.** A candidate who says *"I used index as key at first and it broke when I deleted rows, so I switched to ids"* beats a candidate with a flashier project and no story, every time. Interviewers are listening for evidence you've hit real problems.

Your public repos — the ones worth talking about

Repo	What it demonstrates	Use it to answer
MultistepForm	multi-step state, validation, controlled inputs	"state management", "forms", "a hard bug"
medify	API integration, booking/search flow, real app	"REST APIs", "biggest project"
expenseTracker	CRUD, derived totals, list rendering	"state + derived data"
task-manager	CRUD, filtering, persistence	"component design"
xpagination	pagination logic, edge cases, API data	"pagination", "edge cases"
bot-ai	async flows, streaming/chat UI, loading states	"async", "loading states"
Portfolio / sujith-mondithoka.github.io	responsive layout, deployment, performance	"responsive design", "SEO/perf"
NFT-project / order-summary-component / XIntroSection	pixel-accurate design → code	"translating wireframes" ← *the JD's opening line*

Pick your top 2 and prepare them properly. Mention the rest only in passing.
I'd lead with **medify** (most app-like) and **MultistepForm** (best decision story).

The 2-minute project pitch — the structure to fill in

Use this skeleton for each of your two main projects:

1. **What it is, in one line.** *"Medify is a doctor-appointment booking app — search hospitals by state and city, pick a slot, and it holds your bookings."*
2. **Your stack and why.** *"React with functional components and hooks, React Router for routes, and a public REST API for the hospital data. Styled with [X] because [reason]."*
3. **One hard problem and how you solved it. ← The part they're actually listening for.** *"The search results took a noticeable moment and the UI flashed empty, so I added explicit loading and empty states rather than rendering nothing — and I debounced the input so it wasn't firing a request per keystroke."*
4. **One thing you'd do differently now. ← The maturity signal.** *"I'd move the API calls out of the components into a* `useHospitals` *custom hook — right now the fetch logic is duplicated in two places. And I'd add error handling for a failed request, which I skipped."*

That fourth point is the single highest-leverage sentence in your interview. Self-critique reads as senior. It also lets you control the follow-up: they'll ask about *the thing you already thought through*, not the thing you're weak on.

Prepared answers for the standard project questions

"Walk me through your biggest project." → the 4-part pitch above.

"Why did you choose React for this?"

> "Component reuse and declarative state. The booking list, the card and the form all repeat across pages — writing them once and passing props saved a lot of duplication, and I don't have to manually sync the DOM when the data changes."

"How did you handle state?"

> " `useState` locally, lifted to the nearest common parent when two components needed it. I didn't add Redux — for that size it'd be ceremony. If it grew, I'd reach for Context for the user/session and React Query for server data."

"How did you handle API errors?" → Be honest. If you didn't handle them fully:

> "Only partially — I handled the loading state but not failures. Reading up on it since, I'd wrap the call in try/catch, check `res.ok` because fetch doesn't throw on a 404, and render a retry state." *(Honesty + the correct fix beats bluffing.)*

"How would you make it faster?" → Pull from file 04: image optimisation, code splitting the routes, debouncing the search, virtualising the long list, and measuring with Lighthouse first.

"How would you add tests?"

> "React Testing Library for components — testing what the user sees rather than internal state — and Jest for utility functions. I'd start with the form validation and the booking flow, since those are where a regression actually costs something."

"Did you deploy it?" → Say where (Netlify/Vercel/GitHub Pages) and mention anything you hit: environment variables, build config, routing 404s on refresh with client-side routing. Deployment problems are great, concrete, real-work stories.

• Tonight's homework — do not skip this

1. **Open your top 2 repos and re-read your own code.** Nothing sinks a candidate faster than not recognising code they wrote six months ago.
2. For each, write down: **one decision** you made and **one thing you'd change**.
3. Make sure both **run locally** — `npm install && npm run dev` — and have the deployed links ready to paste.
4. Say the 2-minute pitch out loud. Time it. If it's over 2:30, cut the setup.

Bridging thin experience honestly

If Next.js on your resume is tutorial-level, use this line:

> "My production-style work is React; with Next.js I've built at a project level rather than shipped at scale. But I understand what it's solving — server rendering for SEO and first paint, ISR for content that changes periodically — and I'd be able to contribute quickly."

Then immediately demonstrate it using file 03. Interviewers respect a candidate who marks the boundary of their own knowledge accurately, because it means they can trust everything said inside the boundary. That trust is what gets you hired.

11 · Final Cheat Sheet — TOMORROW MORNING ONLY

07:20 – 08:00. Nothing else. No new material.
If a line here doesn't ring a bell, open that file for 3 minutes and come back.

The 8 sentences that carry this interview

1. **Why Next.js:** *"Plain React ships an empty HTML shell that the browser fills in — bad for first paint and unreliable for crawlers. Next renders on the server, so the browser and Google both get real HTML immediately."*
2. **Rendering strategies:** *"SSG at build, SSR per request, ISR is SSG that regenerates on a timer, CSR in the browser. Bank listing pages → ISR. A student's own application page → SSR or CSR behind auth."*
3. **Server vs Client Components:** *"Server by default, ships zero JS; push* `'use client'` *down to the interactive leaves."*
4. **Core Web Vitals:** *"LCP under 2.5 s — is it there. CLS under 0.1 — does it stay still. INP under 200 ms — does it respond."*
5. **Closure:** *"A function that remembers the scope it was created in. It's how* `useState` *and debounce both work."*
6. **Keys:** *"Stable identity across renders. Index breaks the moment you delete or reorder — React reuses the wrong DOM node."*
7. **Four UI states:** *"Loading, error, empty, success. Empty is the one people forget."*
8. **Business tie-in (say once):** *"Since students find you through Google, server rendering and load speed here are lead generation, not polish."*

JavaScript

- `var` function-scoped · `let` / `const` block-scoped + TDZ · `const` ≠ immutable object
- Event loop: **microtasks (Promises) before macrotasks (setTimeout)** → `1 4 3 2`
- `Promise.all` parallel · `allSettled` all results · `race` first settled · `any` first success
- Arrow functions take `this` from the enclosing scope and can't be rebound
- `??` only falls back on null/undefined; `||` falls back on any falsy (`0`!)
- Spread = **shallow** copy · deep = `structuredClone`
- `fetch` **does not throw on 404/500** — check `res.ok`
- Debounce = after they stop (search) · Throttle = at most once per N ms (scroll)
- `typeof null === 'object'` · `NaN !== NaN` · `Array.isArray()`

React

- Re-render causes: own state · props · **parent re-rendered** · context value
- Reduce: `React.memo` → `useCallback` / `useMemo` → move state down → `children`
- Never mutate state — new reference or no re-render (`Object.is`)
- `setCount(c => c + 1)` when the new value depends on the old
- `useEffect` = sync with the outside world; cleanup `return` ; don't use it for derived data
- `useMemo` value · `useCallback` function · `useRef` persists without re-rendering
- Hooks: top level only — React tracks them **by call order**
- Context re-renders every consumer; memoize the provider value
- `React.lazy` + `Suspense` = code splitting

Next.js

- `app/page.js` routing · `[id]` dynamic · `layout.js` · `loading.js` · `error.js`
- `fetch(url, { next: { revalidate: 60 } })` = ISR · `{ cache: 'no-store' }` = SSR
- `next/image` : WebP/AVIF, srcset, lazy, reserves space (CLS) · `priority` for the LCP image
- `metadata` / `generateMetadata` for per-page title & description
- Hydration = server HTML + client listeners; mismatches come from `Date` , `random` , `window`
- `<Link>` client-side nav + prefetch; `<a>` = full reload
- App Router → `next/navigation` ; Pages Router → `next/router`
- Only `NEXT_PUBLIC_*` reaches the browser — never a secret

CSS

- `box-sizing: border-box` · Flex = 1D, Grid = 2D
- `grid-template-columns: repeat(auto-fit, minmax(260px, 1fr))` = responsive, no media queries
- Mobile-first with `min-width` · viewport meta tag or nothing works
- `display:grid; place-items:center` to centre
- Animate `transform` / `opacity` only — they skip layout and paint

APIs

- 401 = who are you · 403 = I know, still no · 429 = rate limited
- `AbortController` in the effect cleanup → no stale state, no race condition
- CORS is a browser rule, fixed on the **backend**

System design — the 6 step framework

Clarify → Components → State → Data → Edge cases → Performance

- **Never start without asking 2 questions.** "Client side or API filtering?" "How many items?" "Does this need to rank on Google?"
- **Autocomplete:** debounce 400ms · four states · cancel the old request with `AbortController` · cache per query.
- **Listing page:** keep filters **in the URL** so links are shareable, back works, and the server can render it. Reset to page 1 when a filter changes.
- **Multi step form:** all data lives in the **parent**, steps are display only. Otherwise going back destroys what was typed.
- **Reusable component:** props say *what* not *how* · use `children` · spread `...rest` · a component with 15 boolean flags should be split.

Machine coding — the 6 things they're grading

1. `key={item.id}`, never the index
2. Derived data computed during render, not stored in state + effect
3. Empty state handled ("No results for…")
4. Loading **and** error states, ideally with a retry
5. Cleanup — `clearTimeout` / `clearInterval` / `abort`
6. **Talking while you type**

Opening & closing lines

"Tell me about yourself" → Present → Path → **Why here** (file 09 §1). Under 2 min.
Your closing question → *"What's the biggest frontend problem you're hoping this hire helps solve?"*

Filter questions — answer instantly, no hesitation:
18-month commitment ✓ · WFO ✓ · 1st & 3rd Saturdays ✓ · 10–11 AM login ✓
Salary → *"I saw the 4–6 LPA band and I'm comfortable in it; I'd expect to be placed based on how the technical rounds go."* **Never bid below 4.**

The last 5 minutes before you walk in

- Close every tab except your GitHub and your two running projects.
- You know more than you think you do. Nobody expects perfection at this level — they expect someone who reasons clearly and doesn't bluff.
- **If you don't know something: say so, then reason out loud.** That answer has never lost anyone an offer. Bluffing has.
- Sit up. Breathe out slowly. Smile before you speak — it changes your voice.

Go get it.